DSCI 519 · Homework 1

Prompt Engineering for SQL on an Unfamiliar Database

Covers: Session 1 (Chapters 1 to 3). Points: 100. Submit: this notebook (`.ipynb`) with all outputs, plus a PDF export (File > Print > Save as PDF).

The situation

You are the new analyst at Prairie Outfitters, a fictional outdoor retailer with five stores in the Upper Midwest and an online shop. Leadership has questions; the answers are in a SQLite database that no AI model has ever seen. Your job is to use an AI companion effectively and prove that the answers are right.

Rules for AI use in this course

- You may use any AI assistant or API. Using AI is the point of the course.
- You must log what you asked (prompt log), which provider and model answered, and how you verified the result.
- You are responsible for every number you report. "The AI said so" is never a verification.

Grading rubric

Part	What	Points
A	Setup, schema card, and data dictionary context	10
B	Context experiment (no schema vs schema vs schema + dictionary)	20
C	Six business questions with assigned techniques, verified	42 (7 each)
D	Failure log: at least three documented AI failures and fixes	15
E	Reflection memo	13

For each question in Part C, full credit requires: the technique was actually used (prompt shown), the final SQL runs and answers the business question as defined in the data dictionary, and at least one independent verification is shown (a second query, a row-count check, a hand-checked example, or a self-consistency comparison).

In [1]:
```
#@title Setup: install packages and the course helper (run me first)
# code hidden in this copy
```

```
llmkit 1.1.0 is ready.
```

Choose your AI provider

Everything in this course works with any of the providers below. Pick one, make sure its API key is stored in Colab Secrets (click the key icon in the left sidebar, add the secret name shown, and switch on *Notebook access*), and run the cell. You never paste a key into the notebook itself.

`PROVIDER`	Secret name	Cost notes (October 2026, verify before use)	Suggested `MIN_INTERVAL`
`"groq"`	`GROQ_API_KEY`	Free tier, no credit card, generous daily limits	2
`"google"`	`GOOGLE_API_KEY`	Free tier for Flash models (low daily quota; your data may be used for training)	6
`"openai"`	`OPENAI_API_KEY`	Paid, prepaid credit; small models cost cents for this course	0
`"anthropic"`	`ANTHROPIC_API_KEY`	Paid, prepaid credit; Haiku is the low-cost tier	0

`"openrouter"`	`OPENROUTER_API_KEY`	Many models behind one key; some `:free` models	3
`"mistral"`	`MISTRAL_API_KEY`	Free experimentation tier available	2
`"mock"`	none	Offline fake model: returns placeholders, useful only to test that code runs	0

`MODEL = None` uses the provider's default from `llmkit.PROVIDERS`. Model names change frequently; if you get a "model not found" error, run `llm.list_models()` and copy a current name into `MODEL`.

In [2]:
```
from llmkit import LLM, get_secret, estimate_tokens

PROVIDER = "google"      # "groq" | "google" | "openai" | "anthropic" | "openrouter" | "mistral" | "mock"
MODEL = None            # None = provider default, or a specific model id string
MIN_INTERVAL = 2.0      # seconds between calls (protects free-tier rate limits)

llm = LLM(PROVIDER, model=MODEL, min_interval=MIN_INTERVAL, cache=True)
print(llm)
```

```
Paste your GEMINI_API_KEY (input is hidden): ··········
LLM(provider='google', model='gemini-3.5-flash-lite')
```

In [4]:
```
#@title Get the data files
# code hidden in this copy
```

```
Files present: ['prairie_outfitters.db']
```

Part A. Setup and context (10 points)

1. Connect to the database and write `run_sql()` (you may copy it from Session 1).
2. Build a schema card with the `schema_card()` function from Session 1.
3. Read the data dictionary below. It contains business definitions that are *not* visible in the schema.

Prairie Outfitters data dictionary (give this to your AI companion as context)

- stores: physical stores. `opened_date` is an ISO date.
- customers: `loyalty_tier` codes: `B` = Bronze, `S` = Silver, `G` = Gold, `P` = Platinum. Some customers never ordered.
- products: `list_price` is the catalog price; `is_active = 0` means discontinued.
- orders: one row per order. `channel` is `'online'` or `'in_store'`. Online orders have `store_id = NULL`. `status` is `'completed'` or `'cancelled'`; cancelled orders never count as revenue. `order_ts` is local time stored as text `YYYY-MM-DD HH:MM:SS`.
- order_items: one row per product line. The quantity column is `qty`. `unit_price` is the price charged before the line discount; `discount_pct` is a fraction (0.10 = 10% off). Line revenue = `qty * unit_price * (1 - discount_pct)`.
- returns: one row per returned line; `qty_returned` units, `refund_amount` dollars. Refunds are attributed to the period of the *original order*.
- Net revenue = line revenue of completed orders minus refunds on those lines.
- Data covers January 2024 through December 2025. The database is SQLite.

In [5]:
```
import sqlite3, re, pandas as pd
con = sqlite3.connect("hw1/prairie_outfitters.db")

def run_sql(query, limit=30):
    df = pd.read_sql_query(query, con)
    return df.head(limit) if len(df) > limit else df

def schema_card(con):
```

```
    lines = []
    for (t,) in con.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name"):
        cols = con.execute(f"PRAGMA table_info({t})").fetchall()
        lines.append(f"{t}(" + ", ".join(f"{c[1]} {c[2]}" + (" PK" if c[5] else "") for c in cols) + ")")
        for fk in con.execute(f"PRAGMA foreign_key_list({t})").fetchall():
            lines.append(f"  {t}.{fk[3]} -> {fk[2]}.{fk[4]}")
    return "\n".join(lines)

CARD = schema_card(con)
print(CARD)
```

```
customers(customer_id INTEGER PK, first_name TEXT, last_name TEXT, email TEXT, city TEXT, state TEXT, signup_dat
e TEXT, loyalty_tier TEXT)
order_items(order_item_id INTEGER PK, order_id INTEGER, product_id INTEGER, qty INTEGER, unit_price REAL, discou
nt_pct REAL)
  order_items.product_id -> products.product_id
  order_items.order_id -> orders.order_id
orders(order_id INTEGER PK, customer_id INTEGER, store_id INTEGER, channel TEXT, order_ts TEXT, status TEXT, pro
mo_code TEXT)
  orders.store_id -> stores.store_id
  orders.customer_id -> customers.customer_id
products(product_id INTEGER PK, sku TEXT, product_name TEXT, category TEXT, subcategory TEXT, brand TEXT, unit_c
ost REAL, list_price REAL, is_active INTEGER)
returns(return_id INTEGER PK, order_item_id INTEGER, return_ts TEXT, qty_returned INTEGER, reason TEXT, refund_a
mount REAL)
  returns.order_item_id -> order_items.order_item_id
stores(store_id INTEGER PK, store_name TEXT, city TEXT, state TEXT, opened_date TEXT, sq_ft INTEGER)
```

In [7]:
```
DICTIONARY = """
### Prairie Outfitters data dictionary (give this to your AI companion as context)
* **stores**: physical stores. `opened_date` is an ISO date.
* **customers**: `loyalty_tier` codes: `B` = Bronze, `S` = Silver, `G` = Gold, `P` = Platinum. Some customers
never ordered.
* **products**: `list_price` is the catalog price; `is_active = 0` means discontinued.
* **orders**: one row per order. `channel` is ``online`` or ``in_store``. **Online orders have `store_id =
NULL`.**
  `status` is ``completed`` or ``cancelled``; cancelled orders never count as revenue. `order_ts` is local
time stored as
  text `YYYY-MM-DD HH:MM:SS`.
* **order_items**: one row per product line. The quantity column is **`qty`**. `unit_price` is the price
charged before the
  line discount; `discount_pct` is a fraction (0.10 = 10% off).
  **Line revenue** = `qty * unit_price * (1 - discount_pct)`.
* **returns**: one row per returned line; `qty_returned` units, `refund_amount` dollars. Refunds are
attributed to the
  period of the *original order*.
* **Net revenue** = line revenue of completed orders minus refunds on those lines.
* Data covers January 2024 through December 2025. The database is **SQLite**.
"""

def extract_sql(text):
    m = re.search(r"```(?:sql|sqlite)?\s*(.*?)```", text, flags=re.S | re.I)
    return (m.group(1) if m else text).strip()

prompt_log = []
def log(part, technique, prompt, sql, result_note, verification, success):
    prompt_log.append(dict(part=part, technique=technique, provider=llm.provider, model=llm.model,
prompt=prompt,
                           sql=sql, result=result_note, verification=verification, success=success))
```

Part B. The context experiment (20 points)

Business question: What was 2025 net revenue by channel? (Use the definition in the data dictionary.)

Ask the same question three ways, run each SQL, and fill in the table that follows.

1. B1, zero-shot, no context: "Write a SQLite query for Prairie Outfitters' 2025 net revenue by channel."
2. B2, schema only: include the schema card.
3. B3, schema + dictionary: include the schema card and the data dictionary.

Then write your own independent verification query (written or checked by you) and compare all three results with it. Hint: think about cancelled orders, discounts, refunds, and NULL `store_id`s.

In [39]:
```
# B1: zero-shot, no context
#b1_prompt = "Write a SQLite query for Prairie Outfitters' 2025 net revenue by channel. Return only SQL."
#b1_sql = extract_sql(llm.ask(b1_prompt))
#print(b1_sql)
#try:
    #display(run_sql(b1_sql))
#except Exception as exc:
    #print("ERROR:", exc)

# B1: zero-shot, no context
b1_prompt = "Write a SQLite query for Prairie Outfitters' 2025 net revenue by channel. Return only SQL."
b1_sql = extract_sql(llm.ask(b1_prompt))
print(b1_sql)
try:
    display(run_sql(b1_sql))
except Exception as exc:
    print("ERROR:", exc)

log("B1", "zero-shot, no context", b1_prompt, b1_sql,
    "Did not run: no such table OrderItems",
    "Compared table/column names with schema card",
    False)
```

```
SELECT
    c.ChannelName AS Channel,
    SUM(oi.Quantity * oi.UnitPrice * (1 - oi.Discount)) AS NetRevenue
FROM
    Orders o
JOIN
    OrderItems oi ON o.OrderID = oi.OrderID
JOIN
    Channels c ON o.ChannelID = c.ChannelID
WHERE
    strftime('%Y', o.OrderDate) = '2025'
GROUP BY
    c.ChannelName;
ERROR: Execution failed on sql 'SELECT
    c.ChannelName AS Channel,
    SUM(oi.Quantity * oi.UnitPrice * (1 - oi.Discount)) AS NetRevenue
FROM
    Orders o
JOIN
    OrderItems oi ON o.OrderID = oi.OrderID
JOIN
    Channels c ON o.ChannelID = c.ChannelID
WHERE
    strftime('%Y', o.OrderDate) = '2025'
GROUP BY
    c.ChannelName;': no such table: OrderItems
```

In [13]:
```
# B2: schema only (write the prompt)
b2_prompt = f"""You are writing a query for a SQLite database.
```

```
Database schema:
{CARD}

Question: Write a SQLite query for Prairie Outfitters' 2025 net revenue by channel. Return only SQL."""

b2_sql = extract_sql(llm.ask(b2_prompt))
print(b2_sql)
try:
    b2_result = run_sql(b2_sql)
    display(b2_result)
except Exception as exc:
    b2_result = None
    print("ERROR:", exc)

log("B2", "schema only", b2_prompt, b2_sql,
    "Ran: in_store 250,134.02, online 161,490.17",
    "Read SQL against data dictionary; compare with independent verification query",
    False)  # did not exclude cancelled orders
```

```
SELECT
    o.channel,
    SUM((oi.qty * oi.unit_price) * (1.0 - COALESCE(oi.discount_pct, 0.0))) - COALESCE(SUM(r.refund_amount), 0.0)
AS net_revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
LEFT JOIN returns r ON oi.order_item_id = r.order_item_id
WHERE strftime('%Y', o.order_ts) = '2025'
GROUP BY o.channel;
```

	channel	net_revenue
0	in_store	250134.0175
1	online	161490.1700

In [16]:

```
# B3: schema + data dictionary (write the prompt)
b3_prompt = f"""You are writing a query for a SQLite database.

Database schema:
{CARD}

Data dictionary (business rules you must follow):
{DICTIONARY}

Question: Write a SQLite query for Prairie Outfitters' 2025 net revenue by channel. Return only SQL."""

b3_sql = extract_sql(llm.ask(b3_prompt))
print(b3_sql)
try:
    b3_result = run_sql(b3_sql)
    display(b3_result)
except Exception as exc:
    b3_result = None
    print("ERROR:", exc)

log("B3", "schema + data dictionary", b3_prompt, b3_sql,
    "Ran: in_store 238,579.12, online 155,336.65",
    "Compared with independent verification query (revenue and refunds computed separately)",
    True)  # will change to False if verification disagrees
```

```
SELECT
    o.channel,
    SUM(oi.qty * oi.unit_price * (1 - oi.discount_pct)) - COALESCE(SUM(r.refund_amount), 0.0) AS net_revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
LEFT JOIN returns r ON oi.order_item_id = r.order_item_id
WHERE o.status = 'completed'
```

```
  AND strftime('%Y', o.order_ts) = '2025'
GROUP BY o.channel;
```

	channel	net_revenue
0	in_store	238579.115
1	online	155336.650

In [18]:

```
# Your independent verification query (explain in comments why you trust it)
# Why I trust this query:
# 1. It follows the data dictionary step by step:
#    line revenue = qty * unit_price * (1 - discount_pct), completed orders only,
#    minus refunds on those same lines.
# 2. Revenue and refunds are summed in SEPARATE CTEs and only joined at the channel level,
#    so a line with several return rows cannot have its revenue counted more than once.
# 3. The 2025 filter uses a text range on order_ts (works because order_ts is 'YYYY-MM-DD HH:MM:SS'),
#    a different method from the AI's strftime, so the two do not share the same possible mistake.
# 4. The data checks below confirm the assumptions the query depends on.

# ---- Data checks ----
print("Status values:")
display(run_sql("SELECT status, COUNT(*) AS n FROM orders GROUP BY status"))

print("Channel values and store_id rule (online should have NULL store_id):")
display(run_sql("""
SELECT channel,
       COUNT(*) AS n_orders,
       SUM(CASE WHEN store_id IS NULL THEN 1 ELSE 0 END) AS null_store_id
FROM orders
GROUP BY channel
"""))

print("Order lines with a missing discount_pct:")
display(run_sql("SELECT COUNT(*) AS null_discount_lines FROM order_items WHERE discount_pct IS NULL"))

print("Order lines with more than one return row (would cause double counting in a direct join):")
display(run_sql("""
SELECT COUNT(*) AS lines_with_multiple_returns
FROM (SELECT order_item_id FROM returns GROUP BY order_item_id HAVING COUNT(*) > 1)
"""))

# ---- Verification query ----
verify_sql = """
WITH line_revenue AS (
    SELECT o.channel,
           SUM(oi.qty * oi.unit_price * (1 - COALESCE(oi.discount_pct, 0))) AS gross_revenue
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.order_id
    WHERE o.status = 'completed'
      AND o.order_ts >= '2025-01-01' AND o.order_ts < '2026-01-01'
    GROUP BY o.channel
),
line_refunds AS (
    SELECT o.channel,
           SUM(r.refund_amount) AS refunds
    FROM returns r
    JOIN order_items oi ON oi.order_item_id = r.order_item_id
    JOIN orders o ON o.order_id = oi.order_id
    WHERE o.status = 'completed'
      AND o.order_ts >= '2025-01-01' AND o.order_ts < '2026-01-01'
    GROUP BY o.channel
)
SELECT lr.channel,
       ROUND(lr.gross_revenue, 2) AS gross_revenue,
       ROUND(COALESCE(rf.refunds, 0), 2) AS refunds,
       ROUND(lr.gross_revenue - COALESCE(rf.refunds, 0), 2) AS net_revenue
```

```
FROM line_revenue lr
LEFT JOIN line_refunds rf ON rf.channel = lr.channel
ORDER BY lr.channel
"""
verify_result = run_sql(verify_sql)
print("Verification result:")
display(verify_result)

# ---- Compare B2 and B3 with the verification result ----
compare = verify_result[["channel", "net_revenue"]].rename(columns={"net_revenue": "verified"})
for name, res in [("B2", b2_result), ("B3", b3_result)]:
    if res is not None:
        compare = compare.merge(res[["channel", "net_revenue"]].rename(columns={"net_revenue": name}),
                                on="channel", how="left")
        compare[f"{name}_diff"] = (compare[name] - compare["verified"]).round(2)
compare["B1"] = "did not run"
print("Comparison (diff = AI result minus verified):")
display(compare)
```

Status values:

	status	n
0	cancelled	115
1	completed	2139

Channel values and store_id rule (online should have NULL store_id):

	channel	n_orders	null_store_id
0	in_store	1245	0
1	online	1009	1009

Order lines with a missing discount_pct:

	null_discount_lines
0	0

Order lines with more than one return row (would cause double counting in a direct join):

	lines_with_multiple_returns
0	0

Verification result:

	channel	gross_revenue	refunds	net_revenue
0	in_store	254157.30	15578.19	238579.11
1	online	164870.01	9533.36	155336.65

Comparison (diff = AI result minus verified):

	channel	verified	B2	B2_diff	B3	B3_diff	B1
0	in_store	238579.11	250134.0175	11554.91	238579.115	0.01	did not run
1	online	155336.65	161490.1700	6153.52	155336.650	0.00	did not run

B4. Results table and discussion. For B1, B2, and B3 report: did it run, which tables and columns did it invent or miss, did it exclude cancelled orders, apply discounts, subtract refunds, and does the result match your verification? Then, in 150 words or fewer: what does this experiment show about context in prompts?

Your answer here. B4. Results table and discussion.

	B1: zero-shot, no context	B2: schema only	B3: schema + dictionary
Did it run?	No (`no such table: OrderItems`)	Yes	Yes
Invented tables	`OrderItems` , `Channels`	None	None

Invented columns	`Quantity`, `UnitPrice`, `Discount`, `OrderDate`, `OrderID`, `ChannelID`, `ChannelName`	None	None
Missed	Real names `order_items`, `qty`, `unit_price`, `discount_pct`, `order_ts`, `channel`; the `returns` table	The `status` column (no cancelled filter)	Nothing
Excluded cancelled orders?	No	No	Yes
Applied discounts?	Yes (wrong column name)	Yes	Yes
Subtracted refunds?	No	Yes	Yes
Result: in_store / online	No result	$250,134.02$/161,490.17	$238,579.12$/ 155,336.65
Matches verification ($238,579.11$/155,336.65)?	No	No: too high by $11,554.91$/ 6,153.52	Yes (0.01 rounding difference)

My verification query also checked that no order line has more than one return row, so the direct join to `returns` in B2 and B3 did not double count. B2's gap comes only from including cancelled orders.

What this shows about context: Context changed the result from unusable to correct. With no context (B1), the model invented a believable but fictional schema, so the query failed. Adding the schema card (B2) fixed every table and column name, and the query ran, but it still gave the wrong answer: it counted cancelled orders as revenue and overstated total net revenue by about $17,700 (4.5%). A schema shows structure, not business rules. Nothing in it says cancelled orders must be excluded. Only the data dictionary (B3) supplied that rule, and B3 matched my verification. The main lesson is that a query that runs is not the same as a query that is correct. B2 is the most dangerous case because it looks right. Analysts should give the model both the schema and the business definitions, and still verify independently, because context reduces errors but does not prove correctness.

Part C. Six business questions (42 points, 7 each)

For each question: use the assigned technique, show the prompt(s), run the final SQL, perform at least one independent verification, and add one `log(...)` entry. Keep each question in its own cells.

C1. What were the top 5 products by units sold in Q2 2025 (April to June, completed orders), with their category? Break ties alphabetically.

Technique: Few-shot (house style: one CTE, snake_case aliases, ORDER BY the metric)

In [19]:
```
# C1: prompt(s) and final SQL
# Technique: few-shot. Two worked examples show the house style
# (one CTE, snake_case aliases, ORDER BY the metric) before the real question.

c1_examples = """
Example 1
Question: How many completed in-store orders did each store have in 2024?
SQL:
WITH store_orders AS (
    SELECT o.store_id,
           COUNT(*) AS order_count
    FROM orders o
    WHERE o.status = 'completed'
      AND o.channel = 'in_store'
      AND o.order_ts >= '2024-01-01' AND o.order_ts < '2025-01-01'
    GROUP BY o.store_id
)
SELECT s.store_name,
       so.order_count
```

```
FROM store_orders so
JOIN stores s ON s.store_id = so.store_id
ORDER BY so.order_count DESC, s.store_name;

Example 2
Question: What was line revenue by brand for completed orders in Q1 2025?
SQL:
WITH brand_revenue AS (
    SELECT p.brand,
           SUM(oi.qty * oi.unit_price * (1 - oi.discount_pct)) AS line_revenue
    FROM order_items oi
    JOIN orders o ON o.order_id = oi.order_id
    JOIN products p ON p.product_id = oi.product_id
    WHERE o.status = 'completed'
      AND o.order_ts >= '2025-01-01' AND o.order_ts < '2025-04-01'
    GROUP BY p.brand
)
SELECT brand,
       ROUND(line_revenue, 2) AS line_revenue
FROM brand_revenue
ORDER BY line_revenue DESC, brand;
"""

c1_prompt = f"""You are writing SQLite queries for Prairie Outfitters.

Database schema:
{CARD}

Data dictionary (business rules you must follow):
{DICTIONARY}

Follow the house style shown in the examples: exactly one CTE, snake_case aliases, ORDER BY the metric.
{c1_examples}
Now answer this question in the same style.
Question: What were the top 5 products by units sold in Q2 2025 (April to June, completed orders), with their
category? Break ties alphabetically by product name.
Return only SQL."""

c1_sql = extract_sql(llm.ask(c1_prompt))
print(c1_sql)
try:
    c1_result = run_sql(c1_sql)
    display(c1_result)
except Exception as exc:
    c1_result = None
    print("ERROR:", exc)
```

```
WITH product_units AS (
    SELECT p.product_id,
           SUM(oi.qty) AS units_sold
    FROM order_items oi
    JOIN orders o ON o.order_id = oi.order_id
    JOIN products p ON p.product_id = oi.product_id
    WHERE o.status = 'completed'
      AND o.order_ts >= '2025-04-01' AND o.order_ts < '2025-07-01'
    GROUP BY p.product_id
)
SELECT p.product_name,
       p.category,
       pu.units_sold
FROM product_units pu
JOIN products p ON p.product_id = pu.product_id
ORDER BY pu.units_sold DESC, p.product_name ASC
LIMIT 5;
```

product_name	category	units_sold

0	Jig Assortment 100pc	Fishing	48
1	Walleye Pro Spinning Rod	Fishing	48
2	Northern Lights 15F Bag	Camping	35
3	Badlands 4P Tent	Camping	33
4	Ultralight Solo Shelter	Camping	33

In [20]:
```
# C1: verification (and log(...) entry)
# Method: independent calculation in pandas instead of SQL aggregation.
# 1. Pull raw order lines with their order status/date and product info.
# 2. Filter and total units in Python.
# 3. Show ranks 1-8 to check ties at the 5th-place cutoff.
# 4. Compare the top 5 with the AI result programmatically.

raw = pd.read_sql_query("""
SELECT oi.order_item_id, oi.qty, o.status, o.order_ts,
       p.product_id, p.product_name, p.category
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN products p ON p.product_id = oi.product_id
""", con)

# Fan-out check: joining to orders and products should not add rows
n_items = con.execute("SELECT COUNT(*) FROM order_items").fetchone()[0]
print("order_items rows:", n_items, "| rows after joins:", len(raw))

# Product names must be unique, or grouping by name could merge two products
n_names = con.execute("SELECT COUNT(DISTINCT product_name) FROM products").fetchone()[0]
n_products = con.execute("SELECT COUNT(*) FROM products").fetchone()[0]
print("products:", n_products, "| distinct product names:", n_names)

q2 = raw[(raw["status"] == "completed") &
         (raw["order_ts"] >= "2025-04-01") &
         (raw["order_ts"] < "2025-07-01")]
print("Q2 2025 completed order lines:", len(q2))

units = (q2.groupby(["product_id", "product_name", "category"], as_index=False)["qty"].sum()
           .rename(columns={"qty": "units_sold"})
           .sort_values(["units_sold", "product_name"], ascending=[False, True])
           .reset_index(drop=True))
units.index = units.index + 1
print("\nRanks 1-8 (to check ties at the cutoff):")
display(units.head(8))

# Programmatic comparison with the AI result
check = units.head(5)[["product_name", "category", "units_sold"]].reset_index(drop=True)
ai = c1_result[["product_name", "category", "units_sold"]].reset_index(drop=True)
match = check.equals(ai.astype(check.dtypes.to_dict()))
print("AI top 5 matches pandas top 5:", match)

log("C1", "few-shot (house style: one CTE, snake_case, ORDER BY metric)", c1_prompt, c1_sql,
    "Top 5: " + ", ".join(f"{r.product_name} ({r.units_sold})" for r in c1_result.itertuples()),
    "Recomputed in pandas from raw order lines; checked join row counts, unique product names, and ties at
rank 5",
    bool(match))
```

```
order_items rows: 3786 | rows after joins: 3786
products: 31 | distinct product names: 31
Q2 2025 completed order lines: 540

Ranks 1-8 (to check ties at the cutoff):
```

	product_id	product_name	category	units_sold
1	25	Jig Assortment 100pc	Fishing	48

2	23	Walleye Pro Spinning Rod	Fishing	48
3	4	Northern Lights 15F Bag	Camping	35
4	2	Badlands 4P Tent	Camping	33
5	3	Ultralight Solo Shelter	Camping	33
6	28	Filter Straw	Accessories	31
7	27	Insulated Bottle 32oz	Accessories	31
8	7	Pocket Flame Stove	Camping	31

```
AI top 5 matches pandas top 5: True
```

C2. How many distinct customers placed at least one completed order in each month of 2025?

Technique: Chain-of-thought (ask for step-by-step reasoning before the SQL; read the reasoning)

In [21]:
```python
# C2: prompt(s) and final SQL
# Technique: chain-of-thought. The model must reason step by step before writing SQL,
# and we print the full reasoning so we can read and check it.

c2_prompt = f"""You are writing SQLite queries for Prairie Outfitters.

Database schema:
{CARD}

Data dictionary (business rules you must follow):
{DICTIONARY}

Question: How many distinct customers placed at least one completed order in each month of 2025?

Before writing any SQL, think step by step and write out your reasoning:
1. State how you interpret the question and what the output should look like (rows and columns).
2. Which tables and columns are needed, and why.
3. Which filters are needed (order status, date range) and how you will apply them to the order_ts text format.
4. How you will extract the month and count customers without counting anyone twice in the same month.
5. Any edge cases (for example, customers with several orders in one month, or months with no orders).

Then give the final SQLite query in one ```sql code block."""

c2_reply = llm.ask(c2_prompt)
print("=== Full reply (reasoning + SQL) ===")
print(c2_reply)

c2_sql = extract_sql(c2_reply)
print("\n=== Extracted SQL ===")
print(c2_sql)
try:
    c2_result = run_sql(c2_sql)
    display(c2_result)
except Exception as exc:
    c2_result = None
    print("ERROR:", exc)
```

```
=== Full reply (reasoning + SQL) ===
1. **Interpretation of the question and output format:**
   - The question asks for the number of *distinct customers* who placed at least one *completed order* in *each month of 2025*.
   - The output should show the month (e.g., in `YYYY-MM` or month number format) and the count of distinct customers for that month, with one row per month for the year 2025 (January through December).

2. **Tables and columns needed:**
   - `orders`: To get the order timestamp (`order_ts`), customer ID (`customer_id`), and order status (`status
```

`).
   - `customers`: Technically, we can get `customer_id` directly from `orders`, so we may not even need to join `customers` unless filtering requires it. But joining or just using `orders` is sufficient to count distinct customers per month.

3. **Filters needed:**
   - Date range: The year 2025. We can filter using `order_ts LIKE '2025-%'` or `strftime('%Y', order_ts) = '2025'` or `order_ts >= '2025-01-01 00:00:00' AND order_ts < '2026-01-01 00:00:00'`. Using string comparison or `substr(order_ts, 1, 7)` is standard for SQLite.
   - Order status: Only completed orders (`status = 'completed'`). Cancelled orders must be excluded.

4. **Extracting the month and counting distinct customers:**
   - We can extract the month using `substr(order_ts, 1, 7)` (which gives `'YYYY-MM'`).
   - To count unique customers per month without double-counting individuals who placed multiple orders in the same month, we use `COUNT(DISTINCT customer_id)`.

5. **Edge cases:**
   - Multiple orders by the same customer in a single month: Handled correctly by `COUNT(DISTINCT customer_id)`.
   - Months with zero completed orders: The prompt asks "in each month of 2025", but a standard `GROUP BY` query on the `orders` table will only return months that have at least one completed order. If all 12 months are expected, we could use a calendar table or a series, but standard SQL practice for this phrasing groups by the extracted month from the filtered orders. Let's group by `substr(order_ts, 1, 7)`.

```sql
SELECT
    substr(order_ts, 1, 7) AS order_month,
    COUNT(DISTINCT customer_id) AS distinct_customers_count
FROM orders
WHERE status = 'completed'
  AND order_ts >= '2025-01-01 00:00:00'
  AND order_ts < '2026-01-01 00:00:00'
GROUP BY order_month
ORDER BY order_month;
```

=== Extracted SQL ===
```
SELECT
    substr(order_ts, 1, 7) AS order_month,
    COUNT(DISTINCT customer_id) AS distinct_customers_count
FROM orders
WHERE status = 'completed'
  AND order_ts >= '2025-01-01 00:00:00'
  AND order_ts < '2026-01-01 00:00:00'
GROUP BY order_month
ORDER BY order_month;
```

	order_month	distinct_customers_count
0	2025-01	86
1	2025-02	102
2	2025-03	107
3	2025-04	109
4	2025-05	93
5	2025-06	90
6	2025-07	96
7	2025-08	97
8	2025-09	105
9	2025-10	98
10	2025-11	92
11	2025-12	86

In [22]:
```
# C2: verification (and log(...) entry)
# Method: independent calculation in pandas.
# 1. Pull raw completed orders (no SQL grouping).
# 2. Convert order_ts to real dates and take the month from the date, not from the text.
# 3. Count unique customers per month with nunique().
# 4. Hand check: for one month, list the customers with more than one order (should be counted once).
# 5. Compare with the AI result programmatically.

ord_raw = pd.read_sql_query("SELECT order_id, customer_id, status, order_ts FROM orders", con)
ord_raw["order_dt"] = pd.to_datetime(ord_raw["order_ts"], format="%Y-%m-%d %H:%M:%S")

print("Orders with unreadable dates:", ord_raw["order_dt"].isna().sum())
print("Orders with missing customer_id:", ord_raw["customer_id"].isna().sum())

c2_data = ord_raw[(ord_raw["status"] == "completed") & (ord_raw["order_dt"].dt.year == 2025)]
monthly = (c2_data.groupby(c2_data["order_dt"].dt.strftime("%Y-%m"))["customer_id"]
                  .nunique()
                  .rename("verified_customers")
                  .reset_index()
                  .rename(columns={"order_dt": "order_month"}))

# Programmatic comparison with the AI result
c2_compare = monthly.merge(c2_result, on="order_month", how="outer")
c2_compare["diff"] = c2_compare["distinct_customers_count"] - c2_compare["verified_customers"]
print("\nComparison (diff = AI minus verified):")
display(c2_compare)
c2_match = bool((c2_compare["diff"] == 0).all() and len(c2_compare) == 12)
print("All 12 months present and matching:", c2_match)

# Hand check for January: orders vs unique customers
jan = c2_data[c2_data["order_dt"].dt.month == 1]
repeat = jan.groupby("customer_id").size()
print(f"\nJanuary 2025: {len(jan)} completed orders, {jan['customer_id'].nunique()} unique customers, "
      f"{(repeat > 1).sum()} customers with more than one order (each counted once).")

# Other reading of the question, for reference: customers with a completed order in ALL 12 months
active_months = c2_data.groupby("customer_id")["order_dt"].apply(lambda s: s.dt.month.nunique())
print("Other reading - customers with a completed order in all 12 months of 2025:",
      int((active_months == 12).sum()))

log("C2", "chain-of-thought (reasoning before SQL)", c2_prompt, c2_sql,
    "12 monthly counts, from 86 (Jan, Dec) to 109 (Apr)",
    "Recomputed in pandas with real dates and nunique(); hand-checked January repeat customers; "
    "reasoning read and confirmed (did not mention the 'all 12 months' reading)",
    c2_match)
```

```
Orders with unreadable dates: 0
Orders with missing customer_id: 0

Comparison (diff = AI minus verified):
```

	order_month	verified_customers	distinct_customers_count	diff
0	2025-01	86	86	0
1	2025-02	102	102	0
2	2025-03	107	107	0
3	2025-04	109	109	0
4	2025-05	93	93	0
5	2025-06	90	90	0
6	2025-07	96	96	0
7	2025-08	97	97	0
8	2025-09	105	105	0
9	2025-10	98	98	0

10	2025-11	92	92	0
11	2025-12	86	86	0

```
All 12 months present and matching: True

January 2025: 94 completed orders, 86 unique customers, 7 customers with more than one order (each counted onc
e).
Other reading - customers with a completed order in all 12 months of 2025: 0
```

C3. What is the return rate (units returned / units sold, completed orders) by product category, for categories with at least 50 units sold?

Technique: Chain-of-thought + a fan-out check (compare unit totals before and after your joins)

In [25]:
```python
# C3: prompt(s) and final SQL
# Technique: chain-of-thought + fan-out check.
# The model reasons step by step (including fan-out risk) before writing SQL.
# The fan-out check itself (unit totals before vs after joins) is in the verification cell.

c3_prompt = f"""You are writing SQLite queries for Prairie Outfitters.

Database schema:
{CARD}

Data dictionary (business rules you must follow):
{DICTIONARY}

Question: What is the return rate (units returned / units sold, completed orders) by product category,
for categories with at least 50 units sold? Use all data in the database (no date filter).

Before writing any SQL, think step by step and write out your reasoning:
1. State how you interpret the question and what the output should look like (rows and columns).
2. Which tables and columns are needed for units sold and for units returned, and why.
3. Which filters are needed (order status) and where they apply.
4. Fan-out risk: explain whether joining order_items to returns could repeat a line's qty and inflate units
sold,
   and how your query avoids that.
5. How you will apply the "at least 50 units sold" rule and compute the rate without integer division.

Output columns: category, units_sold, units_returned, return_rate (a decimal rounded to 4 places).
Sort by return_rate from highest to lowest.

Then give the final SQLite query in one ```sql code block."""

# ---- Attempt 1: chain-of-thought ----
c3_reply = llm.ask(c3_prompt)
print("=== Attempt 1: full reply (reasoning + SQL) ===")
print(c3_reply)

c3_sql_v1 = extract_sql(c3_reply)
c3_error_v1 = None
try:
    run_sql(c3_sql_v1)
    print("\nAttempt 1 ran.")
except Exception as exc:
    c3_error_v1 = str(exc).split(":")[-1].strip()   # keep only the short error message
    print("\nAttempt 1 ERROR:", c3_error_v1)

# ---- Attempt 2: repair prompt (show the model its own error) ----
c3_repair_prompt = (
    f"Your SQL failed in SQLite with this error: {c3_error_v1}\n"
    "The query contains two SELECT statements and uses an alias that is not defined. "
    "Keep your two-CTE approach (sold and returned) and return ONE corrected, complete SQLite query "
    "in one ```sql code block. No explanation."
```

```
)
if c3_error_v1:
    c3_reply_v2 = llm.chat(
        [{"role": "user", "content": c3_prompt},
         {"role": "assistant", "content": c3_reply},
         {"role": "user", "content": c3_repair_prompt}]
    ).text
    c3_sql = extract_sql(c3_reply_v2)
else:
    c3_sql = c3_sql_v1

print("\n=== Final SQL ===")
print(c3_sql)
try:
    c3_result = run_sql(c3_sql)
    display(c3_result)
except Exception as exc:
    c3_result = None
    print("ERROR:", exc)
```

```
=== Attempt 1: full reply (reasoning + SQL) ===
### Step-by-Step Reasoning

1. **Interpretation and Output**:
   * We need the return rate (total units returned divided by total units sold) for each product category, restr
icted to categories where total units sold is at least 50.
   * Output columns: `category`, `units_sold`, `units_returned`, `return_rate` (rounded to 4 decimal places).
   * Sorting: `return_rate` descending (highest to lowest).

2. **Tables and Columns Needed**:
   * `products`: `product_id`, `category` (to group by product category).
   * `orders`: `order_id`, `status` (to filter for `'completed'` orders).
   * `order_items`: `order_item_id`, `product_id`, `order_id`, `qty` (to compute units sold).
   * `returns`: `order_item_id`, `qty_returned` (to compute units returned).

3. **Filters**:
   * Order status must be `'completed'` (`orders.status = 'completed'`). Cancelled orders and their items do not
count toward units sold or returned.

4. **Fan-Out Risk Mitigation**:
   * An order item can have 0, 1, or multiple return records. If we join `order_items` directly to `returns` in
a single flat query, multiplying rows across joins would inflate `qty` (units sold).
   * To prevent this fan-out, we can compute `units_sold` per category independently from completed `order_items
`, and `units_returned` independently from `returns` (joined to `order_items` and `orders` to ensure they belong
to completed orders), and then join or aggregate these two sets together, OR use correlated subqueries / separat
e CTEs. A clean approach is using two separate CTEs (`sold` and `returned`) and joining them on `category`.

5. **Units Sold, Returned, and Return Rate**:
   * **Units Sold**: Sum of `qty` from `order_items` for completed orders, grouped by product `category`.
   * **Units Returned**: Sum of `qty_returned` from `returns` for order items belonging to completed orders, gro
uped by product `category`. (If an order item has no return, it contributes 0).
   * **Return Rate**: `CAST(units_returned AS REAL) / units_sold` to avoid SQLite integer division. Round to 4 d
ecimal places (`ROUND(..., 4)`).
   * **Filter condition**: `units_sold >= 50` in the final `HAVING` or `WHERE` clause.

---

### Final SQLite Query

```sql
WITH sold AS (
    SELECT
        p.category,
        SUM(oi.qty) AS units_sold
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.order_id
```
```

```
    JOIN products p ON oi.product_id = p.product_id
    WHERE o.status = 'completed'
    GROUP BY p.category
),
returned AS (
    SELECT
        p.category,
        SUM(r.qty_returned) AS units_returned
    FROM returns r
    JOIN order_items oi ON r.order_item_id = oi.order_item_id
    JOIN orders o ON oi.order_id = o.order_id
    JOIN products p ON oi.product_id = p.product_id
    WHERE o.status = 'completed'
    GROUP BY p.category
)
SELECT
    s.category,
    s.units_sold,
    COALESCE(r.units_returned, 0) AS units_returned,
    ROUND(CAST(COALESCE(r.units_returned, 0) AS REAL) / s.units_sold, 4) AS return_rate
FROM sold s
LEFT JOIN returned s_ret ON s.category = s_ret.category
-- Note: using r.units_returned via LEFT JOIN alias 'r'
SELECT
    s.category,
    s.units_sold,
    COALESCE(ret.units_returned, 0) AS units_returned,
    ROUND(CAST(COALESCE(ret.units_returned, 0) AS REAL) / s.units_sold, 4) AS return_rate
FROM sold s
LEFT JOIN returned ret ON s.category = ret.category
WHERE s.units_sold >= 50
ORDER BY return_rate DESC;
```

Attempt 1 ERROR: near "SELECT": syntax error

=== Final SQL ===

```
WITH sold AS (
    SELECT
        p.category,
        SUM(oi.qty) AS units_sold
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.order_id
    JOIN products p ON oi.product_id = p.product_id
    WHERE o.status = 'completed'
    GROUP BY p.category
),
returned AS (
    SELECT
        p.category,
        SUM(r.qty_returned) AS units_returned
    FROM returns r
    JOIN order_items oi ON r.order_item_id = oi.order_item_id
    JOIN orders o ON oi.order_id = o.order_id
    JOIN products p ON oi.product_id = p.product_id
    WHERE o.status = 'completed'
    GROUP BY p.category
)
SELECT
    s.category,
    s.units_sold,
    COALESCE(ret.units_returned, 0) AS units_returned,
    ROUND(CAST(COALESCE(ret.units_returned, 0) AS REAL) / s.units_sold, 4) AS return_rate
FROM sold s
LEFT JOIN returned ret ON s.category = ret.category
```

```
WHERE s.units_sold >= 50
ORDER BY return_rate DESC;
```

	category	units_sold	units_returned	return_rate
0	Footwear	943	113	0.1198
1	Apparel	876	70	0.0799
2	Fishing	763	27	0.0354
3	Accessories	855	27	0.0316
4	Packs	560	17	0.0304
5	Camping	1169	31	0.0265

In [26]:

```
# C3: verification (and log(...) entry)
# 1. Fan-out check: total completed units BEFORE joins vs AFTER each join,
#    plus what a naive single join to returns would give.
# 2. Independent recompute in pandas (rows pulled separately, totals in Python).
# 3. Sanity checks: number of categories, returns larger than units sold, returns on cancelled orders.
# 4. Programmatic comparison with the AI result, then log entries.

# ---- 1. Fan-out check ----
fanout = run_sql("""
SELECT
  (SELECT SUM(oi.qty) FROM order_items oi
     JOIN orders o ON o.order_id = oi.order_id
    WHERE o.status = 'completed')                                    AS units_before_joins,
  (SELECT SUM(oi.qty) FROM order_items oi
     JOIN orders o   ON o.order_id = oi.order_id
     JOIN products p ON p.product_id = oi.product_id
    WHERE o.status = 'completed')                                    AS units_after_products_join,
  (SELECT SUM(oi.qty) FROM order_items oi
     JOIN orders o   ON o.order_id = oi.order_id
     JOIN products p ON p.product_id = oi.product_id
     LEFT JOIN returns r ON r.order_item_id = oi.order_item_id
    WHERE o.status = 'completed')                                    AS units_after_naive_returns_join
""")
print("Fan-out check (all three numbers should be equal):")
display(fanout)
print("Sum of units_sold in AI result:", int(c3_result["units_sold"].sum()))

# ---- 2. Independent recompute in pandas ----
items = pd.read_sql_query("""
SELECT oi.order_item_id, oi.qty, o.status, p.category
FROM order_items oi
JOIN orders o   ON o.order_id = oi.order_id
JOIN products p ON p.product_id = oi.product_id
""", con)
rets = pd.read_sql_query("SELECT order_item_id, qty_returned FROM returns", con)

done = items[items["status"] == "completed"]
sold_pd = done.groupby("category")["qty"].sum().rename("units_sold_v")
ret_pd = (rets.merge(done[["order_item_id", "category"]], on="order_item_id", how="inner")
              .groupby("category")["qty_returned"].sum().rename("units_returned_v"))
c3_check = pd.concat([sold_pd, ret_pd], axis=1).fillna(0).reset_index()
c3_check["return_rate_v"] = (c3_check["units_returned_v"] / c3_check["units_sold_v"]).round(4)
c3_check = c3_check[c3_check["units_sold_v"] >= 50]

# ---- 3. Sanity checks ----
print("\nCategories in products table:",
      con.execute("SELECT COUNT(DISTINCT category) FROM products").fetchone()[0])
print("Return rows where qty_returned > qty sold on that line:", con.execute("""
    SELECT COUNT(*) FROM returns r JOIN order_items oi ON oi.order_item_id = r.order_item_id
    WHERE r.qty_returned > oi.qty""").fetchone()[0])
print("Return rows on cancelled orders (excluded from the rate):", con.execute("""
    SELECT COUNT(*) FROM returns r
```

```
    JOIN order_items oi ON oi.order_item_id = r.order_item_id
    JOIN orders o ON o.order_id = oi.order_id
    WHERE o.status = 'cancelled'""").fetchone()[0])

# ---- 4. Compare with the AI result ----
c3_compare = c3_result.merge(c3_check, on="category", how="outer")
c3_compare["rate_diff"] = (c3_compare["return_rate"] - c3_compare["return_rate_v"]).round(4)
print("\nComparison (AI vs pandas):")
display(c3_compare)
c3_match = bool((c3_compare["units_sold"] == c3_compare["units_sold_v"]).all()
                and (c3_compare["units_returned"] == c3_compare["units_returned_v"]).all()
                and (c3_compare["rate_diff"] == 0).all())
print("AI result matches pandas:", c3_match)

# ---- Log entries ----
log("C3", "chain-of-thought + fan-out check (attempt 1)", c3_prompt, c3_sql_v1,
    f"Did not run: {c3_error_v1}",
    "SQLite error; read SQL and found two SELECT statements and an undefined alias 'r'",
    False)
log("C3", "chain-of-thought + fan-out check (attempt 2, repair prompt)", c3_repair_prompt, c3_sql,
    "6 categories; highest Footwear 0.1198, lowest Camping 0.0265",
    "Fan-out check (unit totals before/after joins); recomputed in pandas; sanity checks on returns",
    c3_match)
```

```
Fan-out check (all three numbers should be equal):
```

	units_before_joins	units_after_products_join	units_after_naive_returns_join
0	5166	5166	5166

```
Sum of units_sold in AI result: 5166

Categories in products table: 6
Return rows where qty_returned > qty sold on that line: 0
Return rows on cancelled orders (excluded from the rate): 0

Comparison (AI vs pandas):
```

	category	units_sold	units_returned	return_rate	units_sold_v	units_returned_v	return_rate_v	rate_diff
0	Accessories	855	27	0.0316	855	27	0.0316	0.0
1	Apparel	876	70	0.0799	876	70	0.0799	0.0
2	Camping	1169	31	0.0265	1169	31	0.0265	0.0
3	Fishing	763	27	0.0354	763	27	0.0354	0.0
4	Footwear	943	113	0.1198	943	113	0.1198	0.0
5	Packs	560	17	0.0304	560	17	0.0304	0.0

```
AI result matches pandas: True
```

C4. How many customers made their first completed order online and later made a completed in-store purchase?

Technique: Tree-of-thought (ask for two approaches, a comparison, and a recommendation)

In [27]:
```
# C4: prompt(s) and final SQL
# Technique: tree-of-thought. The model proposes two different approaches,
# compares them, and recommends one. We run both approaches and the recommended one.

c4_prompt = f"""You are writing SQLite queries for Prairie Outfitters.

Database schema:
{CARD}

Data dictionary (business rules you must follow):
{DICTIONARY}
```

```
Question: How many customers made their first completed order online and later made a completed in-store
purchase?

Work through this as a tree of thought:
1. Approach A: describe one way to solve it, then give the SQL in a ```sql code block.
2. Approach B: describe a genuinely different way (for example, window functions vs. aggregates/EXISTS),
   then give the SQL in a ```sql code block.
3. Comparison: compare A and B on correctness, edge cases, and readability. Consider:
   - only completed orders count (both for "first order" and for the later in-store purchase)
   - "later" means the in-store order's order_ts is strictly after the first order's order_ts
   - a customer could have two orders with the same order_ts
   - customers who never ordered
4. Recommendation: say which approach you recommend and why, then repeat the recommended SQL
   as the LAST ```sql code block.

Each query must return one row with one column named customer_count."""

c4_reply = llm.ask(c4_prompt)
print("=== Full reply (approaches, comparison, recommendation) ===")
print(c4_reply)

# Pull out every ```sql block: expected order is A, B, recommended
c4_blocks = [b.strip() for b in re.findall(r"```(?:sql|sqlite)\s*(.*?)```", c4_reply, flags=re.S | re.I)]
print(f"\nFound {len(c4_blocks)} SQL blocks.")

c4_results = {}
for label, sql in zip(["Approach A", "Approach B", "Recommended"], [c4_blocks[0], c4_blocks[1],
c4_blocks[-1]]):
    try:
        val = run_sql(sql).iloc[0, 0]
    except Exception as exc:
        val = f"ERROR: {exc}"
    c4_results[label] = val
    print(f"{label}: {val}")

c4_sql = c4_blocks[-1]
c4_result = run_sql(c4_sql)
print("\n=== Final (recommended) SQL ===")
print(c4_sql)
display(c4_result)
```

```
=== Full reply (approaches, comparison, recommendation) ===
### Approach A: Using Window Functions (Rank / Row Number)

**Description:**
This approach uses window functions to identify each customer's ordered history chronologically.
1. We first create a CTE or subquery of all completed orders for each customer, using `ROW_NUMBER() OVER (PARTIT
ION BY customer_id ORDER BY order_ts ASC, order_id ASC)` to determine the absolute chronological order of each c
ustomer's completed purchases (breaking timestamp ties with `order_id`).
2. We filter or identify if the order where `row_num = 1` was placed online (`channel = 'online'`).
3. We then check if the same customer has at least one *subsequent* completed order (`row_num > 1`) that was pla
ced in-store (`channel = 'in_store'`) and occurred at or after their first order timestamp (or strictly after, d
epending on tie-handling, though sorting by `order_id` as a tie-breaker prevents exact duplicate timestamps for
the same customer).
4. Finally, we count the distinct customers who satisfy both conditions.

```sql
WITH RankedOrders AS (
    SELECT
        c.customer_id,
        o.channel,
        o.order_ts,
        ROW_NUMBER() OVER (
            PARTITION BY c.customer_id
            ORDER BY o.order_ts ASC, o.order_id ASC
```
```

```
        ) as rn
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    WHERE o.status = 'completed'
)
SELECT COUNT(DISTINCT first_order.customer_id) AS customer_count
FROM RankedOrders first_order
JOIN RankedOrders later_order ON first_order.customer_id = later_order.customer_id
WHERE first_order.rn = 1
  AND first_order.channel = 'online'
  AND later_order.rn > 1
  AND later_order.channel = 'in_store'
  AND later_order.order_ts >= first_order.order_ts;
```

Approach B: Using Conditional Aggregation and `MIN(order_ts)`

Description:
This approach uses basic grouping and aggregation rather than window functions.
1. We determine the exact timestamp and identity of each customer's *first* completed order by grouping by `cust
omer_id` and finding the minimum `order_ts`. Since multiple orders can share the exact same minimum timestamp, w
e use a subquery with `MIN(order_ts)` joined back to orders to grab the channels of those earliest orders.
2. We ensure that *all* of the customer's earliest simultaneous completed orders were online (`channel = 'onlin
e'`). If a customer placed an online order and an in-store order at the exact same millisecond as their very fir
st transaction, they are excluded (or handled cleanly via aggregation).
3. We use an `EXISTS` clause to verify that the customer also has at least one completed order with an `order_ts
` strictly greater than their first order's timestamp where `channel = 'in_store'`.

```sql
WITH FirstOrderTimes AS (
    SELECT
        customer_id,
        MIN(order_ts) AS min_ts
    FROM orders
    WHERE status = 'completed'
    GROUP BY customer_id
),
FirstOrderChannels AS (
    SELECT
        f.customer_id,
        o.channel AS first_channel
    FROM FirstOrderTimes f
    JOIN orders o ON f.customer_id = o.customer_id AND f.min_ts = o.order_ts
    WHERE o.status = 'completed'
)
SELECT COUNT(DISTINCT foc.customer_id) AS customer_count
FROM FirstOrderChannels foc
WHERE foc.first_channel = 'online'
  -- Ensure the customer didn't ALSO have an in-store order at the exact same first timestamp
  AND NOT EXISTS (
      SELECT 1
      FROM FirstOrderChannels foc2
      WHERE foc2.customer_id = foc.customer_id
        AND foc2.first_channel = 'in_store'
  )
  AND EXISTS (
      SELECT 1
      FROM orders o_later
      WHERE o_later.customer_id = foc.customer_id
        AND o_later.status = 'completed'
        AND o_later.channel = 'in_store'
        AND o_later.order_ts > (
            SELECT min_ts FROM FirstOrderTimes WHERE customer_id = foc.customer_id
        )
```

```
  );
```

Comparison

* **Correctness:**
  * *Approach A* strictly sequences every completed order per customer using a deterministic tie-breaker (`order_id`). The first order (`rn = 1`) is definitively the absolute first order. Any subsequent order (`rn > 1`) is definitively later.
  * *Approach B* handles minimum timestamps correctly via `MIN(order_ts)`, but simultaneous orders at the exact same timestamp require careful exclusion logic (`NOT EXISTS` on tied channels) to avoid ambiguity about whether the "first" order was online or in-store.
* **Edge Cases:**
  * Customers with zero orders: Both approaches naturally exclude them (Approach A via inner join/window partitioning on completed orders; Approach B via `FirstOrderTimes` inner join).
  * Customers with multiple orders at the exact same timestamp: Approach A handles this cleanly by imposing a deterministic sequence via `order_id`. Approach B requires extra boilerplate to handle potential channel conflicts at `min_ts`.
* **Readability:**
  * *Approach A* is significantly more readable and maintainable. Window functions make chronological sequencing straightforward without complex nested subqueries and multi-tier CTE joins.

Recommendation

I recommend **Approach A**.

Window functions provide the cleanest and most robust mechanism for tracking chronological sequences of events per entity (customer) in SQL. By incorporating `order_id` into the `ORDER BY` clause of the `ROW_NUMBER()` window function, Approach A elegantly solves the edge case of simultaneous timestamps, ensuring a deterministic "first" and "subsequent" order without writing brittle nested subqueries.

```sql
WITH RankedOrders AS (
    SELECT
        c.customer_id,
        o.channel,
        o.order_ts,
        ROW_NUMBER() OVER (
            PARTITION BY c.customer_id
            ORDER BY o.order_ts ASC, o.order_id ASC
        ) as rn
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    WHERE o.status = 'completed'
)
SELECT COUNT(DISTINCT first_order.customer_id) AS customer_count
FROM RankedOrders first_order
JOIN RankedOrders later_order ON first_order.customer_id = later_order.customer_id
WHERE first_order.rn = 1
  AND first_order.channel = 'online'
  AND later_order.rn > 1
  AND later_order.channel = 'in_store'
  AND later_order.order_ts >= first_order.order_ts;
```

Found 3 SQL blocks.
Approach A: 169
Approach B: 169
Recommended: 169

=== Final (recommended) SQL ===
WITH RankedOrders AS (

```
    SELECT
        c.customer_id,
        o.channel,
        o.order_ts,
        ROW_NUMBER() OVER (
            PARTITION BY c.customer_id
            ORDER BY o.order_ts ASC, o.order_id ASC
        ) as rn
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    WHERE o.status = 'completed'
)
SELECT COUNT(DISTINCT first_order.customer_id) AS customer_count
FROM RankedOrders first_order
JOIN RankedOrders later_order ON first_order.customer_id = later_order.customer_id
WHERE first_order.rn = 1
  AND first_order.channel = 'online'
  AND later_order.rn > 1
  AND later_order.channel = 'in_store'
  AND later_order.order_ts >= first_order.order_ts;
```

	customer_count
0	169

In [31]:

```
# C4: verification (and log(...) entry)
# 1. Independent recompute in pandas: find each customer's first completed order,
#    keep customers whose first order was online, then check for a strictly later completed in-store order.
# 2. Tie check: do any customers have two completed orders with the same order_ts?
#    (This is the only case where the AI's ">=" and the required strict ">" could differ.)
# 3. Rerun the recommended query with ">" to confirm the count does not change.
# 4. Compare with the AI result and log.

c4_orders = pd.read_sql_query("""
SELECT order_id, customer_id, channel, order_ts
FROM orders
WHERE status = 'completed'
""", con)
c4_orders["order_dt"] = pd.to_datetime(c4_orders["order_ts"], format="%Y-%m-%d %H:%M:%S")

# ---- 1. pandas recompute ----
c4_sorted = c4_orders.sort_values(["customer_id", "order_dt", "order_id"])
first = c4_sorted.groupby("customer_id").first().reset_index()[["customer_id", "channel", "order_dt"]]
first = first.rename(columns={"channel": "first_channel", "order_dt": "first_dt"})
print("Customers with at least one completed order:", len(first))
print("Of those, first completed order was online:", (first["first_channel"] == "online").sum())

online_first = first[first["first_channel"] == "online"]
later_store = c4_orders[c4_orders["channel"] == "in_store"].merge(online_first, on="customer_id")
later_store = later_store[later_store["order_dt"] > later_store["first_dt"]]   # strictly later
c4_verified = int(later_store["customer_id"].nunique())
print("Verified count (first order online, later in-store):", c4_verified)

# ---- 2. Tie check ----
ties = c4_orders.groupby(["customer_id", "order_ts"]).size()
n_tie_customers = ties[ties > 1].reset_index()["customer_id"].nunique()
print("\nCustomers with two completed orders at the same timestamp:", n_tie_customers)

# ---- 3. Rerun recommended query with strict ">" ----
c4_sql_strict = c4_sql.replace("later_order.order_ts >= first_order.order_ts", "later_order.order_ts >
first_order.order_ts")
c4_strict = int(run_sql(c4_sql_strict).iloc[0, 0])
print("Recommended query with strict '>':", c4_strict)

# ---- 4. Compare and log ----
c4_ai = int(c4_result.iloc[0, 0])
```

```
c4_a = int(c4_results["Approach A"])
c4_b = int(c4_results["Approach B"])
c4_match = (c4_ai == c4_verified) and (c4_ai == c4_strict) and (c4_a == c4_b == c4_ai)
print(f"\nAI (recommended): {c4_ai} | Approach A: {c4_a} | Approach B: {c4_b} | strict '>': {c4_strict} |
pandas: {c4_verified}")
print("All match:", c4_match)

log("C4", "tree-of-thought (two approaches, comparison, recommendation)", c4_prompt, c4_sql,
    f"{c4_ai} customers (Approaches A and B agree)",
    "Recomputed in pandas; checked same-timestamp ties; reran with strict '>' (AI used '>=' despite the prompt
asking for strictly later)",
    bool(c4_match))
```

```
Customers with at least one completed order: 682
Of those, first completed order was online: 315
Verified count (first order online, later in-store): 169

Customers with two completed orders at the same timestamp: 0
Recommended query with strict '>': 169

AI (recommended): 169 | Approach A: 169 | Approach B: 169 | strict '>': 169 | pandas: 169
All match: True
```

C5. Which store had the highest average order value for completed in-store orders in 2025? Rank all stores.

Technique: Self-consistency (two different formulations; compare results programmatically)

In [32]:

```
# C5: prompt(s) and final SQL
# Technique: self-consistency. The same question is asked in two different formulations;
# both queries are run and their results are compared programmatically.
# Assumption stated in both prompts: order value = sum of line revenue (after discounts, before refunds).

c5_context = f"""You are writing SQLite queries for Prairie Outfitters.

Database schema:
{CARD}

Data dictionary (business rules you must follow):
{DICTIONARY}

Definition for this question: an order's value is the sum of its line revenue
(qty * unit_price * (1 - discount_pct)), before refunds.
"""

# Formulation 1: business wording, model chooses the method
c5_prompt_1 = c5_context + """
Question: Which store had the highest average order value for completed in-store orders in 2025? Rank all
stores.
Output columns: store_rank, store_name, order_count, avg_order_value (rounded to 2 decimals).
Sort by store_rank. Return only SQL in one ```sql code block."""

# Formulation 2: step-by-step wording that describes a different calculation
c5_prompt_2 = c5_context + """
For each store, using only orders where status = 'completed', channel = 'in_store', and order_ts is in 2025:
1. Add up the line revenue of all those orders.
2. Count how many distinct orders there were.
3. Divide step 1 by step 2 to get the average order value.
Then rank the stores from highest to lowest average order value (rank 1 = highest).
Output columns: store_rank, store_name, order_count, avg_order_value (rounded to 2 decimals).
Sort by store_rank. Return only SQL in one ```sql code block."""

c5_results = {}
c5_sqls = {}
for name, prompt in [("F1", c5_prompt_1), ("F2", c5_prompt_2)]:
```

```
    sql = extract_sql(llm.ask(prompt))
    c5_sqls[name] = sql
    print(f"=== {name} SQL ===")
    print(sql)
    try:
        res = run_sql(sql)
        c5_results[name] = res
        display(res)
    except Exception as exc:
        c5_results[name] = None
        print("ERROR:", exc)
    print()

# ---- Programmatic comparison of the two formulations ----
r1, r2 = c5_results["F1"], c5_results["F2"]
if r1 is not None and r2 is not None:
    c5_both = r1.merge(r2, on="store_name", suffixes=("_f1", "_f2"))
    c5_both["aov_diff"] = (c5_both["avg_order_value_f1"] - c5_both["avg_order_value_f2"]).round(2)
    print("Formulation 1 vs Formulation 2:")
    display(c5_both[["store_name", "store_rank_f1", "store_rank_f2", "order_count_f1", "order_count_f2",
                     "avg_order_value_f1", "avg_order_value_f2", "aov_diff"]])
    c5_consistent = (len(r1) == len(r2) == len(c5_both)
                     and (c5_both["aov_diff"] == 0).all()
                     and (c5_both["store_rank_f1"] == c5_both["store_rank_f2"]).all()
                     and (c5_both["order_count_f1"] == c5_both["order_count_f2"]).all())
    print("Formulations agree:", c5_consistent)
else:
    c5_consistent = False
    print("Cannot compare: one formulation failed.")

c5_sql = c5_sqls["F1"]
c5_result = r1
```

```
=== F1 SQL ===
WITH order_values AS (
    SELECT
        o.store_id,
        o.order_id,
        SUM(oi.qty * oi.unit_price * (1 - oi.discount_pct)) AS order_value
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.channel = 'in_store'
      AND o.status = 'completed'
      AND strftime('%Y', o.order_ts) = '2025'
    GROUP BY o.store_id, o.order_id
),
store_stats AS (
    SELECT
        s.store_name,
        COUNT(ov.order_id) AS order_count,
        AVG(ov.order_value) AS avg_order_value
    FROM stores s
    JOIN order_values ov ON s.store_id = ov.store_id
    GROUP BY s.store_id, s.store_name
)
SELECT
    RANK() OVER (ORDER BY avg_order_value DESC) AS store_rank,
    store_name,
    order_count,
    ROUND(avg_order_value, 2) AS avg_order_value
FROM store_stats
ORDER BY store_rank;
```

	store_rank	store_name	order_count	avg_order_value
0	1	Omaha Trailhead	130	390.89
1	2	Rapid City Basecamp	153	367.41

2	3	Sioux City Riverside	146	354.59
3	4	Sioux Falls Flagship	136	342.94
4	5	Fargo Northern	150	324.78

```
=== F2 SQL ===
SELECT
    RANK() OVER (ORDER BY SUM(oi.qty * oi.unit_price * (1 - oi.discount_pct)) / COUNT(DISTINCT o.order_id) DESC)
AS store_rank,
    s.store_name,
    COUNT(DISTINCT o.order_id) AS order_count,
    ROUND(SUM(oi.qty * oi.unit_price * (1 - oi.discount_pct)) / COUNT(DISTINCT o.order_id), 2) AS avg_order_valu
e
FROM stores s
JOIN orders o ON s.store_id = o.store_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.status = 'completed'
  AND o.channel = 'in_store'
  AND strftime('%Y', o.order_ts) = '2025'
GROUP BY s.store_id, s.store_name
ORDER BY store_rank;
```

	store_rank	store_name	order_count	avg_order_value
0	1	Omaha Trailhead	130	390.89
1	2	Rapid City Basecamp	153	367.41
2	3	Sioux City Riverside	146	354.59
3	4	Sioux Falls Flagship	136	342.94
4	5	Fargo Northern	150	324.78

```
Formulation 1 vs Formulation 2:
```

	store_name	store_rank_f1	store_rank_f2	order_count_f1	order_count_f2	avg_order_value_f1	avg_order_value_f2	aov_diff
0	Omaha Trailhead	1	1	130	130	390.89	390.89	0.0
1	Rapid City Basecamp	2	2	153	153	367.41	367.41	0.0
2	Sioux City Riverside	3	3	146	146	354.59	354.59	0.0
3	Sioux Falls Flagship	4	4	136	136	342.94	342.94	0.0
4	Fargo Northern	5	5	150	150	324.78	324.78	0.0

```
Formulations agree: True
```

In [33]:
```
# C5: verification (and log(...) entry)
# 1. Independent recompute in pandas: order value per order, then mean per store.
# 2. Coverage checks: all stores ranked; order counts match the orders table;
#    no completed in-store 2025 order is missing its order lines.
# 3. Show the common mistake (averaging over order LINES instead of orders) for comparison.
# 4. Compare with the AI result and log.

c5_lines = pd.read_sql_query("""
SELECT o.order_id, o.store_id, s.store_name, o.status, o.channel, o.order_ts,
       oi.qty, oi.unit_price, oi.discount_pct
FROM orders o
JOIN stores s ON s.store_id = o.store_id
JOIN order_items oi ON oi.order_id = o.order_id
""", con)
c5_lines["line_revenue"] = c5_lines["qty"] * c5_lines["unit_price"] * (1 - c5_lines["discount_pct"])
c5_f = c5_lines[(c5_lines["status"] == "completed") & (c5_lines["channel"] == "in_store")
                & (c5_lines["order_ts"] >= "2025-01-01") & (c5_lines["order_ts"] < "2026-01-01")]

# ---- 1. pandas recompute ----
per_order = c5_f.groupby(["store_name", "order_id"], as_index=False)["line_revenue"].sum()
c5_check = (per_order.groupby("store_name")
            .agg(order_count_v=("order_id", "nunique"), avg_order_value_v=("line_revenue", "mean"))
            .reset_index())
```

```
c5_check["avg_order_value_v"] = c5_check["avg_order_value_v"].round(2)
c5_check = c5_check.sort_values("avg_order_value_v", ascending=False).reset_index(drop=True)
c5_check["store_rank_v"] = c5_check.index + 1

# ---- 2. Coverage checks ----
print("Stores in stores table:", con.execute("SELECT COUNT(*) FROM stores").fetchone()[0],
      "| stores ranked by AI:", len(c5_result))
n_orders_tbl = con.execute("""
    SELECT COUNT(*) FROM orders
    WHERE status = 'completed' AND channel = 'in_store'
      AND order_ts >= '2025-01-01' AND order_ts < '2026-01-01'""").fetchone()[0]
print("Completed in-store 2025 orders in orders table:", n_orders_tbl,
      "| total order_count in AI result:", int(c5_result["order_count"].sum()))
print("Completed in-store 2025 orders with no order lines:", con.execute("""
    SELECT COUNT(*) FROM orders o
    WHERE o.status = 'completed' AND o.channel = 'in_store'
      AND o.order_ts >= '2025-01-01' AND o.order_ts < '2026-01-01'
      AND NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.order_id = o.order_id)""").fetchone()[0])

# ---- 3. Common mistake: average over order lines ----
wrong = c5_f.groupby("store_name")["line_revenue"].mean().round(2).rename("wrong_avg_per_line")

# ---- 4. Compare with the AI result ----
c5_compare = (c5_result.merge(c5_check, on="store_name", how="outer")
                       .merge(wrong, on="store_name", how="left"))
c5_compare["aov_diff"] = (c5_compare["avg_order_value"] - c5_compare["avg_order_value_v"]).round(2)
print("\nAI (F1) vs pandas, with the line-level mistake shown for reference:")
display(c5_compare[["store_name", "store_rank", "store_rank_v", "order_count", "order_count_v",
                    "avg_order_value", "avg_order_value_v", "aov_diff", "wrong_avg_per_line"]])

c5_match = bool((c5_compare["aov_diff"] == 0).all()
                and (c5_compare["store_rank"] == c5_compare["store_rank_v"]).all()
                and (c5_compare["order_count"] == c5_compare["order_count_v"]).all())
print("AI result matches pandas:", c5_match, "| F1 and F2 agree:", bool(c5_consistent))

top = c5_result.iloc[0]
log("C5", "self-consistency (two formulations compared programmatically)",
    "F1:\n" + c5_prompt_1 + "\n\nF2:\n" + c5_prompt_2, c5_sql,
    f"Highest AOV: {top['store_name']} (${top['avg_order_value']:.2f}); all 5 stores ranked",
    "F1 (AVG of per-order values) and F2 (SUM / COUNT DISTINCT) agree; recomputed in pandas; "
    "checked store coverage, order counts, and orders without lines",
    bool(c5_match and c5_consistent))
```

```
Stores in stores table: 5 | stores ranked by AI: 5
Completed in-store 2025 orders in orders table: 715 | total order_count in AI result: 715
Completed in-store 2025 orders with no order lines: 0

AI (F1) vs pandas, with the line-level mistake shown for reference:
```

	store_name	store_rank	store_rank_v	order_count	order_count_v	avg_order_value	avg_order_value_v	aov_diff	wrong_avg_per_line
0	Fargo Northern	5	5	150	150	324.78	324.78	0.0	191.05
1	Omaha Trailhead	1	1	130	130	390.89	390.89	0.0	244.31
2	Rapid City Basecamp	2	2	153	153	367.41	367.41	0.0	208.97
3	Sioux City Riverside	3	3	146	146	354.59	354.59	0.0	203.82
4	Sioux Falls Flagship	4	4	136	136	342.94	342.94	0.0	193.53

```
AI result matches pandas: True | F1 and F2 agree: True
```

C6. For each loyalty tier (use the full tier names), what is the average number of days between a customer's consecutive completed orders?

Technique: Prompt chaining (plan -> SQL -> senior-engineer review, as three separate calls)

In [34]:

```
# C6: prompt(s) and final SQL
# Technique: prompt chaining as three separate calls.
#   Call 1 (plan)   -> plain-language plan, no SQL
#   Call 2 (SQL)    -> SQL written from the plan
#   Call 3 (review) -> senior-engineer review of the SQL, with verdict and final SQL

c6_context = f"""You are working with a SQLite database for Prairie Outfitters.

Database schema:
{CARD}

Data dictionary (business rules you must follow):
{DICTIONARY}
"""

c6_question = """Question: For each loyalty tier (use the full tier names: Bronze, Silver, Gold, Platinum),
what is the average number of days between a customer's consecutive completed orders?
Definitions for this question:
- Only completed orders count.
- A "gap" is the number of days between one completed order and the same customer's next completed order,
  computed as julianday(next order_ts) - julianday(this order_ts).
- The average for a tier is the mean of ALL gaps from customers in that tier (each gap counts once).
- Customers with fewer than two completed orders have no gaps and do not contribute.
Output columns: loyalty_tier (full name), customers_with_gaps, gap_count, avg_days_between_orders (rounded to
1 decimal).
Sort by avg_days_between_orders ascending."""

# ---- Call 1: plan ----
c6_prompt_plan = c6_context + "\n" + c6_question + """

Write a short numbered plan (5 to 8 steps) in plain language for how to compute this in SQLite.
Name the tables, columns, filters, and functions you will use. Do NOT write any SQL."""
c6_plan = llm.ask(c6_prompt_plan)
print("=== Call 1: plan ===")
print(c6_plan)

# ---- Call 2: SQL from the plan ----
c6_prompt_sql = c6_context + "\n" + c6_question + f"""

Follow this plan exactly:
{c6_plan}

Write the SQLite query. Return only SQL in one ```sql code block."""
c6_sql_draft = extract_sql(llm.ask(c6_prompt_sql))
print("\n=== Call 2: draft SQL ===")
print(c6_sql_draft)
try:
    c6_draft_result = run_sql(c6_sql_draft)
    display(c6_draft_result)
except Exception as exc:
    c6_draft_result = None
    print("ERROR:", exc)

# ---- Call 3: senior-engineer review ----
c6_prompt_review = c6_context + "\n" + c6_question + f"""

You are a senior data engineer reviewing a junior analyst's SQLite query for this question.

Plan the analyst followed:
{c6_plan}

Query to review:
```sql
{c6_sql_draft}
```
```

```
Check carefully:
1. Only completed orders are used, and gaps are between consecutive completed orders of the SAME customer.
2. Each customer's first order (which has no previous order) is excluded from the gaps, not counted as a zero
gap.
3. Days are computed with julianday on order_ts.
4. Tier codes B/S/G/P are mapped to Bronze/Silver/Gold/Platinum.
5. The average is over all gaps in the tier (not an average of per-customer averages).
6. The output columns, rounding, and sort order match the question.
7. The query is valid SQLite.

Write a short list of findings. Then write one line that says exactly "VERDICT: APPROVED" or "VERDICT: CHANGES
NEEDED".
Finally, give the final query (corrected if needed, unchanged if approved) in one ```sql code block."""
c6_review = llm.ask(c6_prompt_review)
print("\n=== Call 3: senior-engineer review ===")
print(c6_review)

c6_review_blocks = re.findall(r"```(?:sql|sqlite)\s*(.*?)```", c6_review, flags=re.S | re.I)
c6_sql = c6_review_blocks[-1].strip() if c6_review_blocks else c6_sql_draft
print("\n=== Final SQL (after review) ===")
print(c6_sql)
try:
    c6_result = run_sql(c6_sql)
    display(c6_result)
except Exception as exc:
    c6_result = None
    print("ERROR:", exc)
```

```
=== Call 1: plan ===
1. Map the loyalty tier codes from the `customers` table to their full names (`B` to Bronze, `S` to Silver, `G`
to Gold, `P` to Platinum) using a `CASE` expression or conditional logic.
2. Select completed orders from the `orders` table by filtering for `status = 'completed'` and join them with th
e `customers` table using `customer_id` to retrieve each customer's loyalty tier.
3. For each customer, assign a sequential order number or retrieve the chronological timestamp of their next com
pleted order using the `LEAD()` window function partitioned by `customer_id` and ordered by `order_ts`.
4. Calculate the time gap in days for each consecutive pair of completed orders by applying `julianday(next_orde
r_ts) - julianday(current_order_ts)`.
5. Exclude customers who have fewer than two completed orders (i.e., where the `LEAD()` function returns a null
value for the next order).
6. Aggregate the data by the full loyalty tier name, counting the number of distinct customers with at least two
completed orders (`customers_with_gaps`), counting the total number of gaps (`gap_count`), and computing the mea
n of the gaps (`avg_days_between_orders`).
7. Round the average days between orders to one decimal place using the `ROUND()` function.
8. Sort the final output by `avg_days_between_orders` in ascending order.

=== Call 2: draft SQL ===
WITH customer_completed_orders AS (
    SELECT
        c.customer_id,
        CASE c.loyalty_tier
            WHEN 'B' THEN 'Bronze'
            WHEN 'S' THEN 'Silver'
            WHEN 'G' THEN 'Gold'
            WHEN 'P' THEN 'Platinum'
            ELSE c.loyalty_tier
        END AS loyalty_tier_name,
        o.order_ts,
        LEAD(o.order_ts) OVER (PARTITION BY c.customer_id ORDER BY o.order_ts) AS next_order_ts
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    WHERE o.status = 'completed'
),
order_gaps AS (
    SELECT
        customer_id,
```

```
        loyalty_tier_name,
        julianday(next_order_ts) - julianday(order_ts) AS gap_days
    FROM customer_completed_orders
    WHERE next_order_ts IS NOT NULL
)
SELECT
    loyalty_tier_name AS loyalty_tier,
    COUNT(DISTINCT customer_id) AS customers_with_gaps,
    COUNT(gap_days) AS gap_count,
    ROUND(AVG(gap_days), 1) AS avg_days_between_orders
FROM order_gaps
GROUP BY loyalty_tier_name
ORDER BY avg_days_between_orders ASC;
```

	loyalty_tier	customers_with_gaps	gap_count	avg_days_between_orders
0	Platinum	30	198	72.4
1	Gold	110	490	98.8
2	Silver	192	462	139.5
3	Bronze	209	307	172.9

=== Call 3: senior-engineer review ===
Here are my findings from reviewing the junior analyst's query:

1. **Filtering Completed Orders**: The CTE correctly filters for `status = 'completed'` on orders before performing window operations.
2. **Window Function & Gaps**: Using `LEAD(o.order_ts) OVER (PARTITION BY c.customer_id ORDER BY o.order_ts)` correctly pairs consecutive completed orders for the same customer, and filtering out `next_order_ts IS NOT NULL` properly removes customers/orders without a subsequent gap.
3. **Date/Time Arithmetic**: `julianday(next_order_ts) - julianday(order_ts)` correctly calculates the day difference in SQLite.
4. **Loyalty Tier Mapping**: The `CASE` statement accurately translates codes ``'B'`, `'S'`, `'G'`, and `'P'` to ``'Bronze'`, `'Silver'`, `'Gold'`, and `'Platinum'`.
5. **Aggregation**: `COUNT(DISTINCT customer_id)`, `COUNT(gap_days)`, and `ROUND(AVG(gap_days), 1)` correctly compute the required statistics across all individual gaps within each tier.
6. **Output & Sorting**: Column aliases (`loyalty_tier`, `customers_with_gaps`, `gap_count`, `avg_days_between_orders`), rounding to 1 decimal place, and sorting in ascending order all match the specification.
7. **SQLite Compatibility**: The syntax is standard and fully compatible with SQLite.

VERDICT: APPROVED

```sql
WITH customer_completed_orders AS (
    SELECT
        c.customer_id,
        CASE c.loyalty_tier
            WHEN 'B' THEN 'Bronze'
            WHEN 'S' THEN 'Silver'
            WHEN 'G' THEN 'Gold'
            WHEN 'P' THEN 'Platinum'
            ELSE c.loyalty_tier
        END AS loyalty_tier_name,
        o.order_ts,
        LEAD(o.order_ts) OVER (PARTITION BY c.customer_id ORDER BY o.order_ts) AS next_order_ts
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    WHERE o.status = 'completed'
),
order_gaps AS (
    SELECT
        customer_id,
        loyalty_tier_name,
        julianday(next_order_ts) - julianday(order_ts) AS gap_days
    FROM customer_completed_orders
    WHERE next_order_ts IS NOT NULL
)
```

```
SELECT
    loyalty_tier_name AS loyalty_tier,
    COUNT(DISTINCT customer_id) AS customers_with_gaps,
    COUNT(gap_days) AS gap_count,
    ROUND(AVG(gap_days), 1) AS avg_days_between_orders
FROM order_gaps
GROUP BY loyalty_tier_name
ORDER BY avg_days_between_orders ASC;
```

```
=== Final SQL (after review) ===
WITH customer_completed_orders AS (
    SELECT
        c.customer_id,
        CASE c.loyalty_tier
            WHEN 'B' THEN 'Bronze'
            WHEN 'S' THEN 'Silver'
            WHEN 'G' THEN 'Gold'
            WHEN 'P' THEN 'Platinum'
            ELSE c.loyalty_tier
        END AS loyalty_tier_name,
        o.order_ts,
        LEAD(o.order_ts) OVER (PARTITION BY c.customer_id ORDER BY o.order_ts) AS next_order_ts
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    WHERE o.status = 'completed'
),
order_gaps AS (
    SELECT
        customer_id,
        loyalty_tier_name,
        julianday(next_order_ts) - julianday(order_ts) AS gap_days
    FROM customer_completed_orders
    WHERE next_order_ts IS NOT NULL
)
SELECT
    loyalty_tier_name AS loyalty_tier,
    COUNT(DISTINCT customer_id) AS customers_with_gaps,
    COUNT(gap_days) AS gap_count,
    ROUND(AVG(gap_days), 1) AS avg_days_between_orders
FROM order_gaps
GROUP BY loyalty_tier_name
ORDER BY avg_days_between_orders ASC;
```

	loyalty_tier	customers_with_gaps	gap_count	avg_days_between_orders
0	Platinum	30	198	72.4
1	Gold	110	490	98.8
2	Silver	192	462	139.5
3	Bronze	209	307	172.9

In [35]:
```
# C6: verification (and log(...) entry)
# 1. Check the loyalty_tier codes actually in the data (any NULLs or unexpected codes?).
# 2. Independent recompute in pandas: sort each customer's completed orders, take the difference to the next
one.
# 3. Hand check: list one customer's orders and gaps.
# 4. Show the other averaging method (mean of per-customer averages) for reference.
# 5. Compare with the AI result and log.

# ---- 1. Tier codes in the data ----
print("loyalty_tier values in customers table:")
display(run_sql("SELECT loyalty_tier, COUNT(*) AS n FROM customers GROUP BY loyalty_tier ORDER BY
loyalty_tier"))

# ---- 2. pandas recompute ----
```

```
c6_orders = pd.read_sql_query("""
SELECT o.customer_id, o.order_id, o.order_ts, c.loyalty_tier
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE o.status = 'completed'
""", con)
c6_orders["order_dt"] = pd.to_datetime(c6_orders["order_ts"], format="%Y-%m-%d %H:%M:%S")
c6_orders = c6_orders.sort_values(["customer_id", "order_dt", "order_id"])
c6_orders["gap_days"] = (c6_orders.groupby("customer_id")["order_dt"].shift(-1)
                         - c6_orders["order_dt"]).dt.total_seconds() / 86400
c6_gaps = c6_orders.dropna(subset=["gap_days"]).copy()
tier_names = {"B": "Bronze", "S": "Silver", "G": "Gold", "P": "Platinum"}
c6_gaps["loyalty_tier"] = c6_gaps["loyalty_tier"].map(tier_names)

c6_check = (c6_gaps.groupby("loyalty_tier")
            .agg(customers_with_gaps_v=("customer_id", "nunique"),
                 gap_count_v=("gap_days", "size"),
                 avg_days_v=("gap_days", "mean"))
            .reset_index())
c6_check["avg_days_v"] = c6_check["avg_days_v"].round(1)

# Gap count rule: gaps = completed orders minus customers (for customers with 2+ orders)
per_cust = c6_orders.groupby("customer_id").size()
print("Expected total gaps (orders - 1 per customer with 2+ orders):", int((per_cust[per_cust >= 2] -
1).sum()),
      "| total gap_count in AI result:", int(c6_result["gap_count"].sum()))

# ---- 3. Hand check one customer ----
sample_id = c6_gaps["customer_id"].iloc[0]
print(f"\nHand check, customer {sample_id}:")
display(c6_orders[c6_orders["customer_id"] == sample_id][["order_id", "order_ts", "gap_days"]])

# ---- 4. Other averaging method, for reference only ----
alt = (c6_gaps.groupby(["loyalty_tier", "customer_id"])["gap_days"].mean()
              .groupby("loyalty_tier").mean().round(1).rename("avg_of_customer_avgs"))

# ---- 5. Compare and log ----
c6_compare = c6_result.merge(c6_check, on="loyalty_tier", how="outer").merge(alt, on="loyalty_tier",
how="left")
c6_compare["avg_diff"] = (c6_compare["avg_days_between_orders"] - c6_compare["avg_days_v"]).round(1)
print("\nAI vs pandas (avg_of_customer_avgs shown for reference only):")
display(c6_compare)

c6_match = bool((c6_compare["avg_diff"] == 0).all()
                and (c6_compare["gap_count"] == c6_compare["gap_count_v"]).all()
                and (c6_compare["customers_with_gaps"] == c6_compare["customers_with_gaps_v"]).all())
print("AI result matches pandas:", c6_match)
print("Reviewer changed the draft SQL:", c6_sql.split() != c6_sql_draft.split())

log("C6", "prompt chaining (plan -> SQL -> senior-engineer review, 3 calls)",
    "PLAN:\n" + c6_prompt_plan + "\n\nSQL:\n" + c6_prompt_sql + "\n\nREVIEW:\n" + c6_prompt_review, c6_sql,
    "; ".join(f"{r.loyalty_tier} {r.avg_days_between_orders} days" for r in c6_result.itertuples()),
    "Recomputed gaps in pandas; checked tier codes and total gap count; hand-checked one customer; reviewer
verdict APPROVED",
    c6_match)
```

loyalty_tier values in customers table:

	loyalty_tier	n
0	B	417
1	G	123
2	P	34
3	S	226

```
Expected total gaps (orders - 1 per customer with 2+ orders): 1457 | total gap_count in AI result: 1457

Hand check, customer 1:
```

	order_id	order_ts	gap_days
21	22	2024-01-13 18:09:27	5.021551
29	30	2024-01-18 18:40:29	308.947025
761	807	2024-11-22 17:24:12	NaN

```
AI vs pandas (avg_of_customer_avgs shown for reference only):
```

	loyalty_tier	customers_with_gaps	gap_count	avg_days_between_orders	customers_with_gaps_v	gap_count_v	avg_days_v	avg_of_customer_avgs	avg_diff
0	Bronze	209	307	172.9	209	307	172.9	179.3	0.0
1	Gold	110	490	98.8	110	490	98.8	111.9	0.0
2	Platinum	30	198	72.4	30	198	72.4	79.8	0.0
3	Silver	192	462	139.5	192	462	139.5	156.0	0.0

```
AI result matches pandas: True
Reviewer changed the draft SQL: False
```

Part D. Failure log (15 points)

Document at least three cases from this assignment where AI-generated SQL was wrong. For each: the prompt, what was wrong (classify it: hallucinated table or column, wrong dialect, wrong business logic, fan-out or double counting, wrong filter or date range, other), how you detected it, and how you fixed it (better prompt, more context, manual edit).

In [41]:
```
#failure_log = [
    # dict(question="B1", prompt="...", problem_type="hallucinated column", what_was_wrong="...",
detected_by="...", fix="...")
#]
#pd.DataFrame(failure_log)


pd.set_option("display.max_colwidth", None)   # show full text in each cell

failure_log = [
    dict(question="B1",
         prompt=b1_prompt,
         problem_type="hallucinated table or column",
         what_was_wrong=("With no context, the model invented tables (OrderItems, Channels) and columns "
                         "(Quantity, UnitPrice, Discount, OrderDate, OrderID, ChannelID, ChannelName). "
                         "It also did not exclude cancelled orders or subtract refunds."),
         detected_by="SQLite error 'no such table: OrderItems'; compared the names with the schema card.",
         fix="More context: added the schema card (B2), then the schema card plus the data dictionary (B3)."),

    dict(question="B2",
         prompt="Schema card + 'Write a SQLite query for Prairie Outfitters' 2025 net revenue by channel.
Return only SQL.' (full text: b2_prompt in prompt log)",
         problem_type="wrong business logic",
         what_was_wrong=("Query ran with correct names, but had no WHERE status = 'completed', so cancelled
orders "
                         "were counted. Net revenue was too high by $11,554.91 (in_store) and $6,153.52
(online)."),
         detected_by=("Independent verification query (revenue and refunds summed in separate CTEs). "
                      "The no-multiple-returns check showed the gap came only from cancelled orders."),
         fix="More context: added the data dictionary (B3), which states that cancelled orders never count. B3
matched the verification."),
```

```
    dict(question="C3 (attempt 1)",
         prompt="Schema card + dictionary + chain-of-thought steps for return rate by category (full text: C3 attempt 1 in prompt log)",
         problem_type="other: invalid SQL (syntax error)",
         what_was_wrong=("The reasoning was correct, but the SQL contained a broken draft SELECT (using an undefined alias 'r') "
                         "followed by a second, corrected SELECT. Two SELECT statements cannot run together."),
         detected_by="SQLite error 'near \"SELECT\": syntax error'; read the SQL and found the leftover draft and the undefined alias.",
         fix=("Better prompt: a repair prompt showed the model its own error and asked for one corrected query "
              "with the same two-CTE approach. The repaired query passed the fan-out check and matched pandas.")),

    dict(question="C4",
         prompt="Schema card + dictionary + tree-of-thought request; prompt said 'later' means strictly after (full text: C4 in prompt log)",
         problem_type="wrong filter or date range",
         what_was_wrong=("The recommended query used later_order.order_ts >= first_order.order_ts instead of the strict '>' "
                         "the prompt asked for. The model's own description admitted 'at or after'. "
                         "It did not change the result here (169), but it could overcount in other data."),
         detected_by=("Read the SQL against the prompt. Checked for customers with two completed orders at the same "
                      "timestamp (0 found) and reran the query with '>' (still 169)."),
         fix="Manual edit: replaced '>=' with '>' in the verification rerun; result unchanged, so the reported answer stands."),
]

pd.DataFrame(failure_log)
```

Out[41]:

	question	prompt	problem_type	what_was_wrong	detected_by	fix
0	B1	Write a SQLite query for Prairie Outfitters' 2025 net revenue by channel. Return only SQL.	hallucinated table or column	With no context, the model invented tables (OrderItems, Channels) and columns (Quantity, UnitPrice, Discount, OrderDate, OrderID, ChannelID, ChannelName). It also did not exclude cancelled orders or subtract refunds.	SQLite error 'no such table: OrderItems'; compared the names with the schema card.	More context: added the schema card (B2), then the schema card plus the data dictionary (B3).
1	B2	Schema card + 'Write a SQLite query for Prairie Outfitters' 2025 net revenue by channel. Return only SQL.' (full text: b2_prompt in prompt log)	wrong business logic	Query ran with correct names, but had no WHERE status = 'completed', so cancelled orders were counted. Net revenue was too high by $11,554.91 (in_store) and 6,153.52 (online).	Independent verification query (revenue and refunds summed in separate CTEs). The no-multiple-returns check showed the gap came only from cancelled orders.	More context: added the data dictionary (B3), which states that cancelled orders never count. B3 matched the verification.
2	C3 (attempt 1)	Schema card + dictionary + chain-of-thought steps for return rate by category (full text: C3 attempt 1 in prompt log)	other: invalid SQL (syntax error)	The reasoning was correct, but the SQL contained a broken draft SELECT (using an undefined alias 'r') followed by a second, corrected SELECT. Two SELECT statements cannot run together.	SQLite error 'near "SELECT": syntax error'; read the SQL and found the leftover draft and the undefined alias.	Better prompt: a repair prompt showed the model its own error and asked for one corrected query with the same two-CTE approach. The repaired query passed the fan-out check and matched pandas.
3	C4	Schema card + dictionary + tree-of-thought request; prompt said 'later' means strictly after (full text: C4 in prompt log)	wrong filter or date range	The recommended query used later_order.order_ts >= first_order.order_ts instead of the strict '>' the prompt asked for. The model's own description admitted 'at or after'. It did not change the result here (169), but it could overcount in other data.	Read the SQL against the prompt. Checked for customers with two completed orders at the same timestamp (0 found) and reran the query with '>' (still 169).	Manual edit: replaced '>=' with '>' in the verification rerun; result unchanged, so the reported answer stands.

In [43]:
```
#pd.DataFrame(prompt_log)
# Prompt log: every AI attempt, with provider, model, prompt, SQL, result, verification, and success.
# The schema card and data dictionary appear in almost every prompt, so they are shown as labels
# to keep the table readable. Their full text is printed in Part A.

log_df = pd.DataFrame(prompt_log)
log_df["prompt"] = (log_df["prompt"]
                    .str.replace(CARD, "[SCHEMA CARD]", regex=False)
                    .str.replace(DICTIONARY, "[DATA DICTIONARY]", regex=False))

# Remove duplicate rows created by rerunning a cell, then sort by part (B1, B2, ..., C6)
before = len(log_df)
log_df = log_df.drop_duplicates(subset=["part", "technique", "sql"])
log_df = log_df.sort_values(["part", "technique"], kind="stable").reset_index(drop=True)
print(f"Log entries: {len(log_df)} (removed {before - len(log_df)} duplicates from reruns)")
print("Successful:", int(log_df["success"].sum()), "| Failed:", int((~log_df["success"].astype(bool)).sum()))
print("Parts covered:", ", ".join(log_df["part"].unique()))

log_df
```

```
Log entries: 10 (removed 3 duplicates from reruns)
Successful: 7 | Failed: 3
Parts covered: B1, B2, B3, C1, C2, C3, C4, C5, C6
```

Out[43]:

	part	technique	provider	model	prompt	sql	result	verification	success
0	B1	zero-shot, no context	google	gemini-3.5-flash-lite	Write a SQLite query for Prairie Outfitters' 2025 net revenue by channel. Return only SQL.	SELECT \n c.ChannelName AS Channel,\n SUM(oi.Quantity * oi.UnitPrice * (1 - oi.Discount)) AS NetRevenue\nFROM \n Orders o\nJOIN \n OrderItems oi ON o.OrderID = oi.OrderID\nJOIN \n Channels c ON o.ChannelID = c.ChannelID\nWHERE \n strftime('%Y', o.OrderDate) = '2025'\nGROUP BY \n c.ChannelName;	Did not run: no such table OrderItems	Compared table/column names with schema card	False
1	B2	schema only	google	gemini-3.5-flash-lite	You are writing a query for a SQLite database.\n\nDatabase schema:\n[SCHEMA CARD]\n\nQuestion: Write a SQLite query for Prairie Outfitters' 2025 net revenue by channel. Return only SQL.	SELECT \n o.channel,\n SUM((oi.qty * oi.unit_price) * (1.0 - COALESCE(oi.discount_pct, 0.0))) - COALESCE(SUM(r.refund_amount), 0.0) AS net_revenue\nFROM orders o\nJOIN order_items oi ON o.order_id = oi.order_id\nLEFT JOIN returns r ON	Ran: in_store 250,134.02, online 161,490.17	Read SQL against data dictionary; compare with independent	False

						oi.order_item_id = r.order_item_id\nWHERE strftime('%Y', o.order_ts) = '2025'\nGROUP BY o.channel;		verification query	
2	B3	schema + data dictionary	google	gemini-3.5-flash-lite	You are writing a query for a SQLite database.\n\nDatabase schema:\n[SCHEMA CARD]\n\nData dictionary (business rules you must follow):\n[DATA DICTIONARY]\n\nQuestion: Write a SQLite query for Prairie Outfitters' 2025 net revenue by channel. Return only SQL.	SELECT \n o.channel,\n SUM(oi.qty * oi.unit_price * (1 - oi.discount_pct)) - COALESCE(SUM(r.refund_amount), 0.0) AS net_revenue\nFROM orders o\nJOIN order_items oi ON o.order_id = oi.order_id\nLEFT JOIN returns r ON oi.order_item_id = r.order_item_id\nWHERE o.status = 'completed'\n AND strftime('%Y', o.order_ts) = '2025'\nGROUP BY o.channel;	Ran: in_store 238,579.12, online 155,336.65	Computed with independent verification query (revenue and refunds computed separately)	True
3	C1	few-shot (house style: one CTE, snake_case, ORDER BY metric)	google	gemini-3.5-flash-lite	You are writing SQLite queries for Prairie Outfitters.\n\nDatabase schema:\n[SCHEMA CARD]\n\nData dictionary (business rules you must follow):\n[DATA DICTIONARY]\n\nFollow the house style shown in the examples: exactly one CTE, snake_case aliases, ORDER BY the metric.\n\nExample 1\nQuestion: How many completed in-store orders did each store have in 2024?\nSQL:\nWITH store_orders AS (\n SELECT o.store_id,\n COUNT(*) AS order_count\n FROM orders o\n WHERE o.status = 'completed'\n AND o.channel = 'in_store'\n AND o.order_ts >= '2024-01-01' AND o.order_ts < '2025-01-01'\n GROUP BY o.store_id\n)\nSELECT s.store_name,\n so.order_count\nFROM store_orders so\nJOIN stores s ON s.store_id = so.store_id\nORDER BY so.order_count DESC, s.store_name;\n\nExample 2\nQuestion: What was line revenue by brand for completed orders in Q1 2025?\nSQL:\nWITH brand_revenue AS (\n SELECT p.brand,\n SUM(oi.qty * oi.unit_price * (1 - oi.discount_pct)) AS line_revenue\n FROM order_items oi\n JOIN orders o ON o.order_id = oi.order_id\n JOIN products p ON p.product_id = oi.product_id\n WHERE o.status = 'completed'\n AND o.order_ts >= '2025-01-01' AND o.order_ts < '2025-04-01'\n GROUP BY p.brand\n)\nSELECT brand,\n ROUND(line_revenue, 2) AS line_revenue\nFROM brand_revenue\nORDER BY line_revenue DESC, brand;\n\nNow answer this question in the same style.\nQuestion: What were the top 5 products by units sold in Q2 2025 (April to June, completed orders), with their category? Break ties alphabetically by product name.\nReturn only SQL.	WITH product_units AS (\n SELECT p.product_id,\n SUM(oi.qty) AS units_sold\n FROM order_items oi\n JOIN orders o ON o.order_id = oi.order_id\n JOIN products p ON p.product_id = oi.product_id\n WHERE o.status = 'completed'\n AND o.order_ts >= '2025-04-01' AND o.order_ts < '2025-07-01'\n GROUP BY p.product_id\n)\nSELECT p.product_name,\n p.category,\n pu.units_sold\nFROM product_units pu\nJOIN products p ON p.product_id = pu.product_id\nORDER BY pu.units_sold DESC, p.product_name ASC\nLIMIT 5;	Top 5: Jig Assortment 100pc (48), Walley Pro Spinning Rod (48), Northern Lights 15 Bag (35), Badlands 4P Tent (33), Ultr	Recomputed in pandas from raw order lines; checked join row counts, unique product names, and ties at rank 5	True

								alight Solo Shelter (33)	
4	C2	chain-of-thought (reasoning before SQL)	google	gemini-3.5-flash-lite	You are writing SQLite queries for Prairie Outfitters.\n\nDatabase schema:\n[SCHEMA CARD]\n\nData dictionary (business rules you must follow):\n[DATA DICTIONARY]\n\nQuestion: How many distinct customers placed at least one completed order in each month of 2025?\n\nBefore writing any SQL, think step by step and write out your reasoning:\n1. State how you interpret the question and what the output should look like (rows and columns).\n2. Which tables and columns are needed, and why.\n3. Which filters are needed (order status, date range) and how you will apply them to the order_ts text format.\n4. How you will extract the month and count customers without counting anyone twice in the same month.\n5. Any edge cases (for example, customers with several orders in one month, or months with no orders).\n\nThen give the final SQLite query in one ```sql code block.	SELECT \n substr(order_ts, 1, 7) AS order_month,\n COUNT(DISTINCT customer_id) AS distinct_customers_count\nFROM orders\nWHERE status = 'completed'\n AND order_ts >= '2025-01-01 00:00:00'\n AND order_ts < '2026-01-01 00:00:00'\nGROUP BY order_month\nORDER BY order_month;	12 monthly counts, from 86 (Jan, Dec) to 109 (Apr)	Recomputed in pandas with real dates and nunique(); hand-checked January repeat customers; reasoning read and confirmed (did not mention the 'all 12 months' reading)	True
5	C3	chain-of-thought + fan-out check (attem	google	gemini-3.5-flash-lite	You are writing SQLite queries for Prairie Outfitters.\n\nDatabase schema:\n[SCHEMA CARD]\n\nData dictionary (business rules you must follow):\n[DATA DICTIONARY]\n\nQuestion: What is the return rate (units returned / units sold, completed orders) by product category,\nfor categories with at least 50 units sold? Use all data in the database (no date filter).\n\nBefore writing any SQL, think step by step and write out your reasoning:\n1. State how you interpret the question and what the output should look like (rows and columns).\n2. Which tables and columns are needed for units sold and for units returned, and why.\n3. Which filters are needed (order status) and where they apply.\n4. Fan-out risk: explain whether joining order_items to returns could repeat a line's qty and inflate units sold,\n and how your query avoids that.\n5. How you will apply the "at least 50 units sold" rule and compute the rate without integer division.\n\nOutput columns: category, units_sold, units_returned, return_rate (a decimal rounded to 4 places).\nSort by return_rate from highest to lowest.\n\nThen give the final SQLite query in one ```sql code block.	WITH sold AS (\n SELECT \n p.category,\n SUM(oi.qty) AS units_sold\n FROM order_items oi\n JOIN orders o ON oi.order_id = o.order_id\n JOIN products p ON oi.product_id = p.product_id\n WHERE o.status = 'completed'\n GROUP BY p.category\n),\n returned AS (\n SELECT \n	Did not run: near "SELECT": syntax error	SQLite error; read SQL and found two SELECT statements and	False

		pt 1)			p.category,\n SUM(r.qty_returned) AS units_returned\n FROM returns r\n JOIN order_items oi ON r.order_item_id = oi.order_item_id\n JOIN orders o ON oi.order_id = o.order_id\n JOIN products p ON oi.product_id = p.product_id\n WHERE o.status = 'completed'\n GROUP BY p.category\n)\nSELECT \n s.category,\n s.units_sold,\n COALESCE(r.units_returned, 0) AS units_returned,\n ROUND(CAST(COALESCE(r.units_returned, 0) AS REAL) / s.units_sold, 4) AS return_rate\nFROM sold s\nLEFT JOIN returned s_ret ON s.category = s_ret.category\n-- Note: using r.units_returned via LEFT JOIN alias 'r'\nSELECT \n s.category,\n s.units_sold,\n COALESCE(ret.units_returned, 0) AS units_returned,\n ROUND(CAST(COALESCE(ret.units_returned, 0) AS REAL) / s.units_sold, 4) AS return_rate\nFROM sold s\nLEFT JOIN returned ret ON s.category = ret.category\nWHERE s.units_sold >= 50\nORDER BY return_rate DESC;	an undefined alias 'r'	
6	C3	chain-of-thought + fa	google	gemini-3.5-flas	Your SQL failed in SQLite with this error: near "SELECT": syntax error\nThe query contains two SELECT statements and uses an alias that is not defined. Keep your two-CTE approach (sold and returned) and return ONE corrected, complete SQLite query in one ```sql code block. No explanation.	WITH sold AS (\n SELECT \n p.category,\n SUM(oi.qty) AS units_sold\n FROM order_items oi\n JOIN orders o ON oi.order_id = o.order_id\n	6 categories; highest Footwe	Fan-out check (unit totals before	True

		n-out check (attempt 2, repair prompt)	h-lite		JOIN products p ON oi.product_id = p.product_id\n WHERE o.status = 'completed'\n GROUP BY p.category\n),\n returned AS (\n SELECT \n p.category,\n SUM(r.qty_returned) AS units_returned\n FROM returns r\n JOIN order_items oi ON r.order_item_id = oi.order_item_id\n JOIN orders o ON oi.order_id = o.order_id\n JOIN products p ON oi.product_id = p.product_id\n WHERE o.status = 'completed'\n GROUP BY p.category\n)\nSELECT \n s.category,\n s.units_sold,\n COALESCE(ret.units_returned, 0) AS units_returned,\n ROUND(CAST(COALESCE(ret.units_returned, 0) AS REAL) / s.units_sold, 4) AS return_rate\nFROM sold s\nLEFT JOIN returned ret ON s.category = ret.category\nWHERE s.units_sold >= 50\nORDER BY return_rate DESC;	ar 0.1 198, lowest Camping 0.0265	/after joins); recomputed in pandas; sanity checks on returns
7	C4	tree-of-thought (two approaches, comparison, reco	google	gemini-3.5-flash-lite	You are writing SQLite queries for Prairie Outfitters.\n\nDatabase schema:\n[SCHEMA CARD]\n\nData dictionary (business rules you must follow):\n[DATA DICTIONARY]\n\nQuestion: How many customers made their first completed order online and later made a completed in-store purchase?\n\nWork through this as a tree of thought:\n1. Approach A: describe one way to solve it, then give the SQL in a ```sql code block.\n2. Approach B: describe a genuinely different way (for example, window functions vs. aggregates/EXISTS),\n then give the SQL in a ```sql code block.\n3. Comparison: compare A and B on correctness, edge cases, and readability. Consider:\n - only completed orders count (both for "first order" and for the later in-store purchase)\n - "later" means the in-store order's order_ts is strictly after the first order's order_ts\n - a customer could have two orders with the same order_ts\n - customers who never ordered\n4. Recommendation: say which approach you recommend and why, then repeat the recommended SQL\n as the LAST ```sql code block.\n\nEach query must return one row with one column named customer_count.	WITH RankedOrders AS (\n SELECT \n c.customer_id,\n o.channel,\n o.order_ts,\n ROW_NUMBER() OVER (\n PARTITION BY c.customer_id\n ORDER BY o.order_ts ASC, o.order_id ASC\n) as rn\n FROM customers c\n JOIN orders o ON c.customer_id = o.customer_id\n WHERE o.status = 'completed'\n)\nSELECT	169 customers (Approaches A and B agree)	Recomputed in pandas; checked same-timestamp ties; rerun with strict '>	True

							mmendation)	COUNT(DISTINCT first_order.customer_id) AS customer_count \nFROM RankedOrders first_order\nJOIN RankedOrders later_order ON first_order.customer_id = later_order.customer_id\nWHERE first_order.rn = 1 \n AND first_order.channel = 'online'\n AND later_order.rn > 1 \n AND later_order.channel = 'in_store'\n AND later_order.order_ts >= first_order.order_ts;	(Aused '>= ' despite the prompt asking for strictly later)	
8	C5	self-consistency (two formulations compared programmatically)	google	gemini-3.5-flash-lite	F1:\nYou are writing SQLite queries for Prairie Outfitters.\n\nDatabase schema:\n[SCHEMA CARD]\n\nData dictionary (business rules you must follow):\n[DATA DICTIONARY]\n\nDefinition for this question: an order's value is the sum of its line revenue\n(qty * unit_price * (1 - discount_pct)), before refunds.\n\nQuestion: Which store had the highest average order value for completed in-store orders in 2025? Rank all stores.\nOutput columns: store_rank, store_name, order_count, avg_order_value (rounded to 2 decimals).\nSort by store_rank. Return only SQL in one ```sql code block.\n\nF2:\nYou are writing SQLite queries for Prairie Outfitters.\n\nDatabase schema:\n[SCHEMA CARD]\n\nData dictionary (business rules you must follow):\n[DATA DICTIONARY]\n\nDefinition for this question: an order's value is the sum of its line revenue\n(qty * unit_price * (1 - discount_pct)), before refunds.\n\nFor each store, using only orders where status = 'completed', channel = 'in_store', and order_ts is in 2025:\n1. Add up the line revenue of all those orders.\n2. Count how many distinct orders there were.\n3. Divide step 1 by step 2 to get the average order value.\nThen rank the stores from highest to lowest average order value (rank 1 = highest).\nOutput columns: store_rank, store_name, order_count, avg_order_value (rounded to 2 decimals).\nSort by store_rank. Return only SQL in one ```sql code block.	WITH order_values AS (\n SELECT\n o.store_id,\n o.order_id,\n SUM(oi.qty * oi.unit_price * (1 - oi.discount_pct)) AS order_value\n FROM orders o\n JOIN order_items oi ON o.order_id = oi.order_id\n WHERE o.channel = 'in_store'\n AND o.status = 'completed'\n AND strftime('%Y', o.order_ts) = '2025'\n GROUP BY o.store_id, o.order_id\n),\n store_stats AS (\n SELECT\n s.store_name,\n COUNT(ov.order_id) AS order_count,\n AVG(ov.order_value) AS avg_order_value\n FROM stores s\n JOIN order_values ov ON s.store_id = ov.store_id\n GROUP BY s.store_id, s.store_name\n)\nSELECT\n RANK() OVER (ORDER BY avg_order_value DESC) AS store_rank,\n store_name,\n order_count,\n	Highest AOV: Omaha Trailhead ($390.89); all 5 stores ranked	F1 (AVG of per-order values) and F2 (SUM / COUNT DISTINCT) agree; recomputed in pandas; checked store coverage, order counts, and orders without lines	True

					ROUND(avg_order_value, 2) AS avg_order_value \nFROM store_stats\nORDER BY store_rank;			
96	C6	prompt chaining (plan -> SQL -> senior-engineer review, 3 calls)	google	gemini-3.5-flash-lite	PLAN:\nYou are working with a SQLite database for Prairie Outfitters.\n\nDatabase schema:\n[SCHEMA CARD]\n\nData dictionary (business rules you must follow):\n[DATA DICTIONARY]\n\nQuestion: For each loyalty tier (use the full tier names: Bronze, Silver, Gold, Platinum),\nwhat is the average number of days between a customer's consecutive completed orders?\nDefinitions for this question:\n- Only completed orders count.\n- A "gap" is the number of days between one completed order and the same customer's next completed order,\n computed as julianday(next order_ts) - julianday(this order_ts).\n- The average for a tier is the mean of ALL gaps from customers in that tier (each gap counts once).\n- Customers with fewer than two completed orders have no gaps and do not contribute.\nOutput columns: loyalty_tier (full name), customers_with_gaps, gap_count, avg_days_between_orders (rounded to 1 decimal).\nSort by avg_days_between_orders ascending.\n\nWrite a short numbered plan (5 to 8 steps) in plain language for how to compute this in SQLite.\nName the tables, columns, filters, and functions you will use. Do NOT write any SQL.\n\nSQL:\nYou are working with a SQLite database for Prairie Outfitters.\n\nDatabase schema:\n[SCHEMA CARD]\n\nData dictionary (business rules you must follow):\n[DATA DICTIONARY]\n\nQuestion: For each loyalty tier (use the full tier names: Bronze, Silver, Gold, Platinum),\nwhat is the average number of days between a customer's consecutive completed orders?\nDefinitions for this question:\n- Only completed orders count.\n- A "gap" is the number of days between one completed order and the same customer's next completed order,\n computed as julianday(next order_ts) - julianday(this order_ts).\n- The average for a tier is the mean of ALL gaps from customers in that tier (each gap counts once).\n- Customers with fewer than two completed orders have no gaps and do not contribute.\nOutput columns: loyalty_tier (full name), customers_with_gaps, gap_count, avg_days_between_orders (rounded to 1 decimal).\nSort by avg_days_between_orders ascending.\n\nFollow this plan exactly:\n1. Map the loyalty tier codes from the `customers` table to their full names (`B` to Bronze, `S` to Silver, `G` to Gold, `P` to Platinum) using a `CASE` expression or conditional logic.\n2. Select completed orders from the `orders` table by filtering for `status = 'completed'` and join them with the `customers` table using `customer_id` to retrieve each customer's loyalty tier.\n3. For each customer, assign a sequential order number or retrieve the chronological timestamp of their next completed order using the `LEAD()` window function partitioned by `customer_id` and ordered by `order_ts`.\n4. Calculate the time gap in days for each consecutive pair of completed orders by applying `julianday(next_order_ts) - julianday(current_order_ts)`.\n5. Exclude customers who have fewer than two completed orders (i.e., where the `LEAD()` function returns a null value for the next order).\n6. Aggregate the data by the full loyalty tier name, counting the number of distinct customers with at least two completed orders (`customers_with_gaps`), counting the total number of gaps (`gap_count`), and computing the mean of the gaps (`avg_days_between_orders`).\n7. Round the average days between orders to one decimal place using the `ROUND()` function.\n8. Sort the final output by `avg_days_between_orders` in ascending order.\n\nWrite the SQLite query. Return only SQL in one ```sql code block.\n\nREVIEW:\nYou are working with a SQLite database for Prairie Outfitters.\n\nDatabase schema:\n[SCHEMA CARD]\n\nData dictionary (business rules you must follow):\n[DATA DICTIONARY]\n\nQuestion: For each loyalty tier (use the full tier names: Bronze, Silver, Gold, Platinum),\nwhat is the average number of days between a customer's consecutive completed orders?\nDefinitions for this question:\n- Only completed orders count.\n- A "gap" is the number of days between one completed order and the same customer's next completed order,\n computed as julianday(next order_ts) - julianday(this order_ts).\n- The average for a tier is the mean of ALL gaps from customers in that tier (each gap counts once).\n- Customers with fewer than two completed orders have no gaps and do not contribute.\nOutput columns: loyalty_tier (full name), customers_with_gaps, gap_count, avg_days_between_orders (rounded to 1 decimal).\nSort by avg_days_between_orders ascending.\n\nYou are a senior data engineer reviewing a junior analyst's SQLite query for this question.\n\nPlan the analyst followed:\n1. Map the loyalty tier codes from the `customers` table to their full names (`B` to Bronze, `S` to Silver, `G` to Gold, `P` to Platinum) using a `CASE` expression or conditional logic.\n2. Select completed orders from the `orders` table by filtering for `status = 'completed'` and join them with the `customers` table using `customer_id` to retrieve each customer's loyalty tier.\n3. For each customer, assign a sequential order number or retrieve the chronological timestamp of their next completed order using the `LEAD()` window function partitioned by `customer_id` and ordered by `order_ts`.\n4. Calculate the time gap in days for each consecutive pair of completed orders by applying `julianday(next_order_ts) - julianday(current_order_ts)`.\n5. Exclude customers who have fewer than two completed orders (i.e., where the `LEAD()` function returns a null value for the next order).\n6. Aggregate the data by the full loyalty tier name, counting the number of distinct customers with at least two completed orders (`customers_with_gaps`), counting the total number of gaps (`gap_count`), and computing the mean of the gaps (`avg_days_between_orders`).\n7. Round the average days between orders to one decimal place using the `ROUND()` function.\n8. Sort the final output by `avg_days_between_orders` in ascending order.\n\nQuery to review:\n```sql\nWITH customer_completed_orders AS (\n SELECT \n c.customer_id,\n CASE c.loyalty_tier\n WHEN 'B' THEN 'Bronze'\n WHEN 'S' THEN 'Silver'\n WHEN 'G' THEN 'Gold'\n WHEN 'P' THEN 'Platinum'\n ELSE c.loyalty_tier\n END AS loyalty_tier_name,\n o.order_ts,\n LEAD(o.order_ts) OVER (PARTITION BY c.customer_id ORDER BY o.order_ts) AS next_order_ts\n FROM customers c\n JOIN orders o ON c.customer_id = o.customer_id\n WHERE o.status = 'completed'\n),\norder_gaps AS (\n SELECT \n customer_id,\n loyalty_tier_name,\n julianday(next_order_ts) - julianday(order_ts) AS gap_days\n FROM customer_completed_orders\n WHERE next_order_ts IS NOT NULL\n)\nSELECT \n loyalty_tier_name AS loyalty_tier,\n COUNT(DISTINCT customer_id) AS customers_with_gaps,\n COUNT(gap_days) AS gap_count,\n ROUND(AVG(gap_days), 1) AS avg_days_between_orders\nFROM order_gaps\nGROUP BY loyalty_tier_name\nORDER BY avg_days_between_orders	WITH customer_completed_orders AS (\n SELECT \n c.customer_id,\n CASE c.loyalty_tier\n WHEN 'B' THEN 'Bronze'\n WHEN 'S' THEN 'Silver'\n WHEN 'G' THEN 'Gold'\n WHEN 'P' THEN 'Platinum'\n ELSE c.loyalty_tier\n END AS loyalty_tier_name,\n o.order_ts,\n LEAD(o.order_ts) OVER (PARTITION BY c.customer_id ORDER BY o.order_ts) AS next_order_ts\n FROM customers c\n JOIN orders o ON c.customer_id = o.customer_id\n WHERE o.status = 'completed'\n),\norder_gaps AS (\n SELECT \n customer_id,\n loyalty_tier_name,\n julianday(next_order_ts) - julianday(order_ts) AS gap_days\n FROM customer_completed_orders\n WHERE next_order_ts IS NOT NULL\n)\nSELECT \n loyalty_tier_name AS loyalty_tier,\n COUNT(DISTINCT customer_id) AS customers_with_gaps,\n COUNT(gap_days) AS gap_count,\n ROUND(AVG(gap_days), 1) AS avg_days_between_orders\nFROM order_gaps\nGROUP BY loyalty_tier_nam	Platinum 72.4 days; Gold 98.8 days; Silver 139.5 days; Bronze 172.9 days	Recomputed gaps in pandas; checked tier codes and total gap count; reviewer verdict APPROVED	True

```
ASC;\n```\n\nCheck carefully:\n1. Only completed orders are used, and gaps are between consecutive completed orders of the SAME customer.\n2. Each customer's first order (which has no previous order) is excluded from the gaps, not counted as a zero gap.\n3. Days are computed with julianday on order_ts.\n4. Tier codes B/S/G/P are mapped to Bronze/Silver/Gold/Platinum.\n5. The average is over all gaps in the tier (not an average of per-customer averages).\n6. The output columns, rounding, and sort order match the question.\n7. The query is valid SQLite.\n\nWrite a short list of findings. Then write one line that says exactly "VERDICT: APPROVED" or "VERDICT: CHANGES NEEDED".\nFinally, give the final query (corrected if needed, unchanged if approved) in one ```sql code block.
```

```
e\nORDER BY avg_days_between_orders ASC;
```

Part E. Reflection memo (13 points)

In 250 to 350 words, write a memo to the Prairie Outfitters analytics manager answering: *Under what conditions should analysts be allowed to report numbers from AI-generated SQL?* Address context (schema, definitions), verification practices, documentation (prompt logs), and one risk that verification cannot fully remove.

To: Analytics Manager, Prairie Outfitters

From: Sharon Wamalwa, Analyst

Subject: Conditions for reporting numbers from AI-generated SQL

Recommendation. Analysts should report numbers from AI-generated SQL only when four conditions are met: the model received full context, the result was independently verified, the work is logged, and any definition choices are stated with the number.

1. Context. The model must receive the schema card and the data dictionary. In my test, the model with no context invented tables and the query failed. With the schema only, the query ran but counted cancelled orders, overstating 2025 net revenue by about $17,700 (4.5%). Only the schema plus the dictionary gave the correct answer. A query that runs is not proof that it is correct.

2. Verification. Every reported number needs at least one check that does not reuse the AI's query: a second query written by the analyst, a recomputation in Python, a fan-out or row-count check, or a hand-checked example. In this assignment, verification caught a syntax error, the cancelled-order mistake, and a query that used "on or after" when the question said "strictly after".

3. Documentation. Each number needs a prompt log entry: provider and model, prompt, final SQL, result, and verification method. This lets a reviewer repeat the work and see which attempts failed.

4. Stated assumptions. Where the dictionary does not define a term, the analyst must write down the definition used. For example, I measured average order value before refunds, and averaged all gaps between orders rather than each customer's average. That second choice changed the results by 6 to 17 days per tier.

Remaining risk. Verification cannot fully remove the risk that the analyst and the AI share the same wrong understanding of the question. My checks confirm that the SQL matches my definition, not that my definition matches what leadership meant. If both are wrong in the same way, the numbers agree and are still wrong. The protection is to confirm key definitions with the business owner before numbers are published.

In [ ]: